



数字逻辑与处理器基础

Fundamentals of Digital Logic and Processor



第五讲 计算机指令集系统 (1/2)

李学清

助教：陈仲豪*/唐文骏/曾令安

#腾讯会议：453-1488-6548

*本节内容负责答疑助教

2024年10月21日

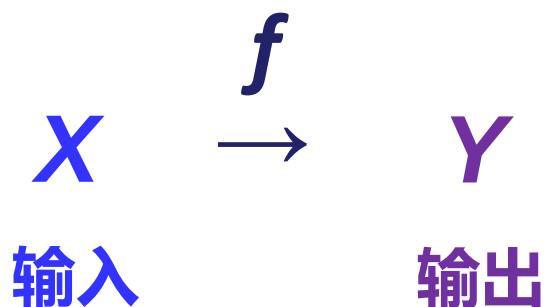
除法器大作业与第一次习题课

- 除法器大作业（10月21日发布，见网络学堂）
 - DDL: 23:59, 11月30日
 - 不要抄袭或迟交
 - 按照说明文档要求，电子版提交网络学堂
- 第一次习题课
 - 时间：10月26日（周六）19:00 – 21:00
 - 地点：系馆3层报告厅
 - 内容：期中复习，数字逻辑部分回顾与部分习题讲解
 - 课后进行答疑，录屏将上传至网络学堂

期中考试

- 时间：11月3日（周日）上午9:00 – 11:00
- 地点：TBD
- 范围：数字逻辑部分
 - 数的表示与布尔函数，组合逻辑，时序逻辑
 - 不包括指令系统和微处理器
- 如有时间冲突，请课下立即联系助教，并email抄送任课老师
 - xueqingli@tsinghua.edu.cn
- 期中考试考前答疑安排
 - 10月26日（周六）19:00 – 21:00，习题课后，系馆3层报告厅
 - 11月2日（周六）19:00 – 21:00，系馆9-206

在时序逻辑之后.....



周	日期	暂定主要内容
1	9/9	绪论
2	9/21	布尔代数 (中秋节调休)
3	9/23	组合逻辑
4	9/30	时序逻辑
5	10/7	国庆节放假
6	10/14	时序逻辑
7	10/21	计算机指令系统
8	11/3	期中考试
9	11/4	计算机指令系统
10	11/11	微处理器
11	11/18	微处理器
12	11/25	流水线
13	12/2	流水线
13	12/7	流水线/存储器 (调课)
15	12/16	存储器
16	12/23	存储器/IO与总线/总结

指令集系统学习目标

- 知识点：指令集的概念、原理和评价指标
 - RISC/CISC、存储结构、基础计算机系统、基础指令、ISA、CPI、执行时间
- 能力
 - 掌握分析和设计指令集的能力
 - 掌握评价计算机性能的方法
- 思想
 - 通用同步计算
 - 系统的跨层次协同设计与优化

本节课目录

- 引言
 - 从算法到处理器：过渡
 - 通用处理器的结构
 - 程序执行流程
- 指令集设计
- 总结

用数字电路解决实际问题

- 时序逻辑电路设计方法



$$f : x \rightarrow y$$

① 专用芯片

不同算法 → 不同电路结构

② 通用CPU

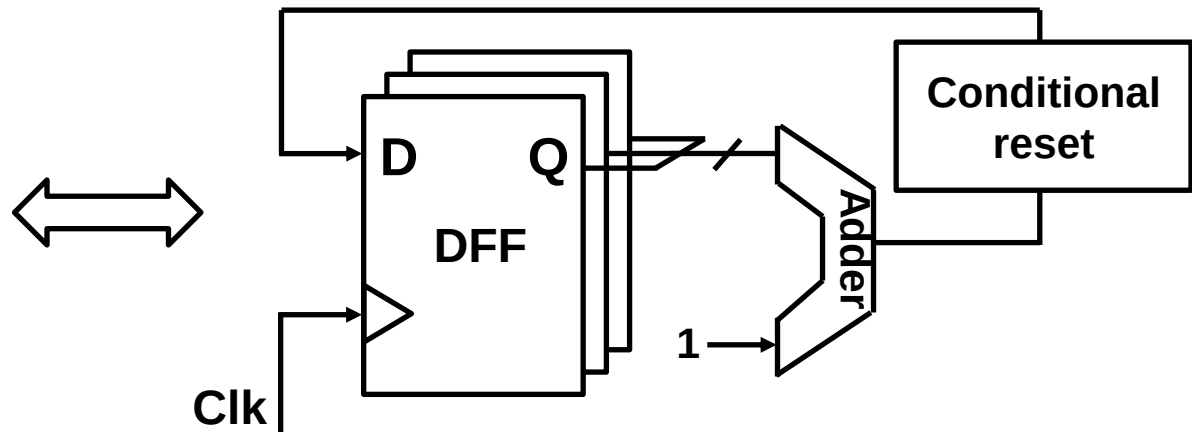
不同算法 → 不同指令流



解决问题：专用芯片 vs 通用CPU

例：计数器

```
int cnt = 0;
for (i=0;;i++)
{
    cnt += 1;
    if(cnt > 7)
        cnt = 0;
}
```



- 问题：如何支持通用算法？

一些软件算法实例

顺序执行，除非跳转

QUICK_SORT(A, p, r)

```
if  $p < r$ 
   $q = \text{PARTITION}(A, p, r)$ 
   $\text{QUICK\_SORT}(A, p, q - 1)$ 
   $\text{QUICK\_SORT}(A, q + 1, r)$ 
```

PARTITION(A, p, r)

```
 $i = p - 1$ 
for  $j = p$  to  $r - 1$ 
  if  $A[j] < A[r]$ 
     $i = i + 1$ 
    exchange  $A[j]$  with  $A[i]$ 
exchange  $A[i + 1]$  with  $A[r]$ 
return  $i + 1$ 
```

DIJKSTRA(G, w, s)

```
 $S = \emptyset$ 
 $Q = G.V$ 
while  $Q \neq \emptyset$ 
   $u = \text{EXTRACT-MIN}(Q)$ 
   $S = S \cup \{u\}$ ;
  for each  $v \in Q.\text{Adj}(u)$ 
     $\text{RELAX}(u, v, w)$ 
```

RELAX(u, v, w)

```
if  $v.d > u.d + w(u, v)$ 
   $v.d = u.d + w(u, v)$ 
   $v.\pi = u$ 
```

不同点 & 相同点?

一些软件算法实例

顺序执行，除非跳转

QUICK_SORT(A, p, r)

if $p < r$

$q = \text{PARTITION}(A, p, r)$

QUICK_SORT($A, p, q - 1$)

QUICK_SORT($A, q + 1, r$)

PARTITION(A, p, r)

$i = p - 1$

for $j = p$ to $r - 1$

if $A[j] < A[r]$

$i = i + 1$

exchange $A[j]$ with $A[i]$

exchange $A[i + 1]$ with $A[r]$

return $i + 1$

赋值

DIJKSTRA(G, w, s)

$S = \emptyset$

$Q = G.V$

while $Q \neq \emptyset$

$u = \text{EXTRACT-MIN}(Q)$

$S = S \cup \{u\};$

for each $v \in Q.\text{Adj}(u)$

RELAX(u, v, w)

RELAX(u, v, w)

if $v.d > u.d + w(u, v)$

$v.d = u.d + w(u, v)$

$v.\pi = u$

不同点 & 相同点?

一些软件算法实例

顺序执行，除非跳转

QUICK_SORT(A, p, r)

```
if  $p < r$   
   $q = \text{PARTITION}(A, p, r)$   
  QUICK_SORT( $A, p, q - 1$ )  
  QUICK_SORT( $A, q + 1, r$ )
```

PARTITION(A, p, r)

```
 $i = p - 1$   
for  $j = p$  to  $r - 1$   
  if  $A[j] < A[r]$   
     $i = i + 1$   
    exchange  $A[j]$  with  $A[i]$   
exchange  $A[i + 1]$  with  $A[r]$   
return  $i + 1$ 
```

赋值

算术操作
逻辑操作

DIJKSTRA(G, w, s)

```
 $S = \emptyset$   
 $Q = G.V$   
while  $Q \neq \emptyset$   
   $u = \text{EXTRACT-MIN}(Q)$   
   $S = S \cup \{u\};$   
  for each  $v \in Q.\text{Adj}(u)$   
    RELAX( $u, v, w$ )
```

RELAX(u, v, w)

```
if  $v.d > u.d + w(u, v)$   
   $v.d = u.d + w(u, v)$   
   $v.\pi = u$ 
```

不同点 & 相同点?

一些软件算法实例

顺序执行，除非跳转

QUICK_SORT(A, p, r)

if $p < r$

$q = \text{PARTITION}(A, p, r)$

$\text{QUICK_SORT}(A, p, q - 1)$

$\text{QUICK_SORT}(A, q + 1, r)$

PARTITION(A, p, r)

$i = p - 1$

for $j = p$ to $r - 1$

 if $A[j] < A[r]$

$i = i + 1$

 exchange $A[j]$ with $A[i]$

exchange $A[i + 1]$ with $A[r]$

return $i + 1$

赋值

算术操作
逻辑操作

分支跳转

DIJKSTRA(G, w, s)

$S = \emptyset$

$Q = G.V$

while $Q \neq \emptyset$

$u = \text{EXTRACT-MIN}(Q)$

$S = S \cup \{u\};$

 for each $v \in Q.\text{Adj}(u)$

$\text{RELAX}(u, v, w)$

RELAX(u, v, w)

if $v.d > u.d + w(u, v)$

$v.d = u.d + w(u, v)$

$v.\pi = u$

不同点 & 相同点?

一些软件算法实例

顺序执行，除非跳转

QUICK_SORT(A, p, r)

if $p < r$

$q = \text{PARTITION}(A, p, r)$
 $\text{QUICK_SORT}(A, p, q - 1)$
 $\text{QUICK_SORT}(A, q + 1, r)$

PARTITION(A, p, r)

$i = p - 1$

for $j = p$ to $r - 1$

if $A[j] < A[r]$

$i = i + 1$

exchange $A[j]$ with $A[i]$

exchange $A[i + 1]$ with $A[r]$

return $i + 1$

赋值

算术操作
逻辑操作

分支跳转
直接跳转
过程调用

DIJKSTRA(G, w, s)

$S = \emptyset$

$Q = G.V$

while $Q \neq \emptyset$

$u = \text{EXTRACT-MIN}(Q)$

$S = S \cup \{u\};$

for each $v \in Q.\text{Adj}(u)$

$\text{RELAX}(u, v, w)$

RELAX(u, v, w)

if $v.d > u.d + w(u, v)$

$v.d = u.d + w(u, v)$

$v.\pi = u$

不同点 & 相同点?

一些软件算法实例

顺序执行，除非跳转

QUICK_SORT(A, p, r)

```
if  $p < r$ 
   $q = \text{PARTITION}(A, p, r)$ 
   $\text{QUICK\_SORT}(A, p, q - 1)$ 
   $\text{QUICK\_SORT}(A, q + 1, r)$ 
```

PARTITION(A, p, r)

```
 $i = p - 1$ 
for  $j = p$  to  $r - 1$ 
  if  $A[j] < A[r]$ 
     $i = i + 1$ 
    exchange  $A[j]$  with  $A[i]$ 
exchange  $A[i + 1]$  with  $A[r]$ 
return  $i + 1$ 
```

赋值

算术操作

逻辑操作

分支跳转

直接跳转

过程调用

访存操作

DIJKSTRA(G, w, s)

```
 $S = \emptyset$ 
 $Q = G.V$ 
while  $Q \neq \emptyset$ 
   $u = \text{EXTRACT-MIN}(Q)$ 
   $S = S \cup \{u\};$ 
  for each  $v \in Q.\text{Adj}(u)$ 
     $\text{RELAX}(u, v, w)$ 
```

RELAX(u, v, w)

```
if  $v.d > u.d + w(u, v)$ 
   $v.d = u.d + w(u, v)$ 
   $v.\pi = u$ 
```

不同点 & 相同点?

一些软件算法实例

顺序执行，除非跳转

QUICK_SORT(A, p, r)

```
if  $p < r$ 
   $q = \text{PARTITION}(A, p, r)$ 
  QUICK_SORT( $A, p, q - 1$ )
  QUICK_SORT( $A, q + 1, r$ )
```

PARTITION(A, p, r)

```
 $i = p - 1$ 
for  $j = p$  to  $r - 1$ 
  if  $A[j] < A[r]$ 
     $i = i + 1$ 
    exchange  $A[j]$  with  $A[i]$ 
exchange  $A[i + 1]$  with  $A[r]$ 
return  $i + 1$ 
```

赋值

算术操作

逻辑操作

分支跳转

直接跳转

过程调用

访存操作

DIJKSTRA(G, w, s)

```
 $S = \emptyset$ 
 $Q = G.V$ 
while  $Q \neq \emptyset$ 
   $u = \text{EXTRACT-MIN}(Q)$ 
   $S = S \cup \{u\};$ 
  for each  $v \in Q.\text{Adj}(u)$ 
    RELAX( $u, v, w$ )
```

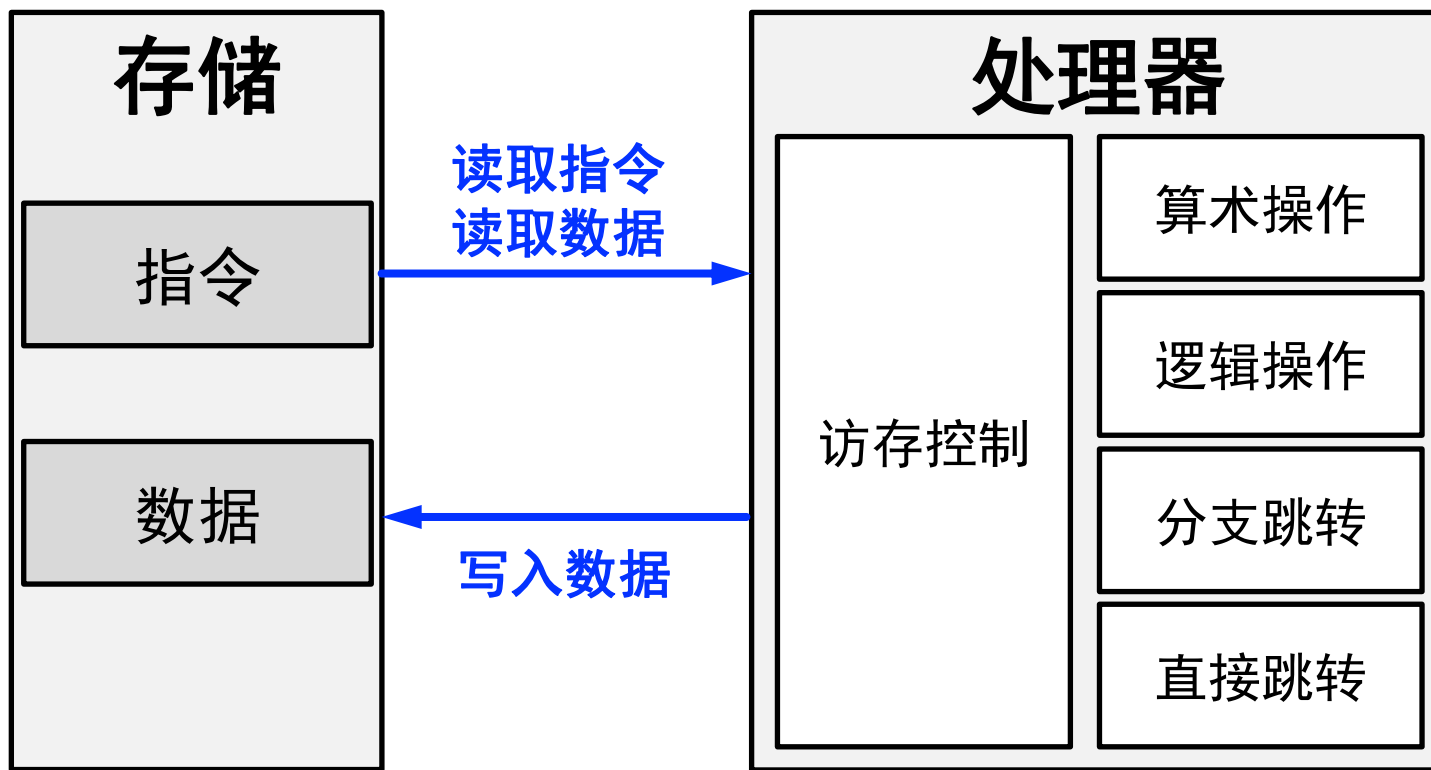
RELAX(u, v, w)

```
if  $v.d > u.d + w(u, v)$ 
   $v.d = u.d + w(u, v)$ 
   $v.\pi = u$ 
```

算法本质上是**几种类型的指令**组成的序列

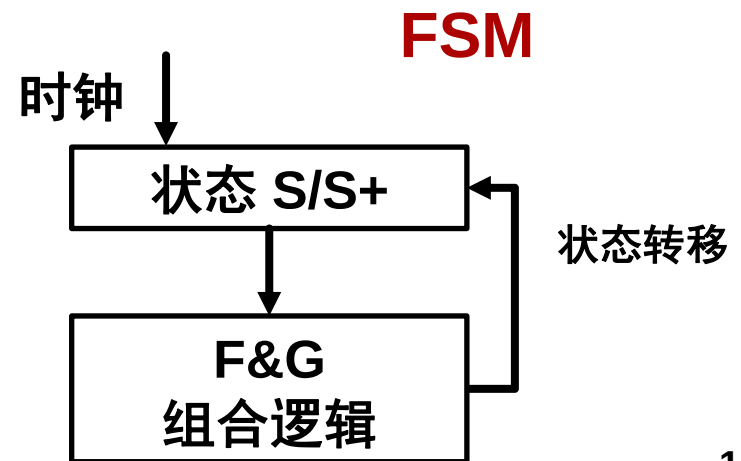
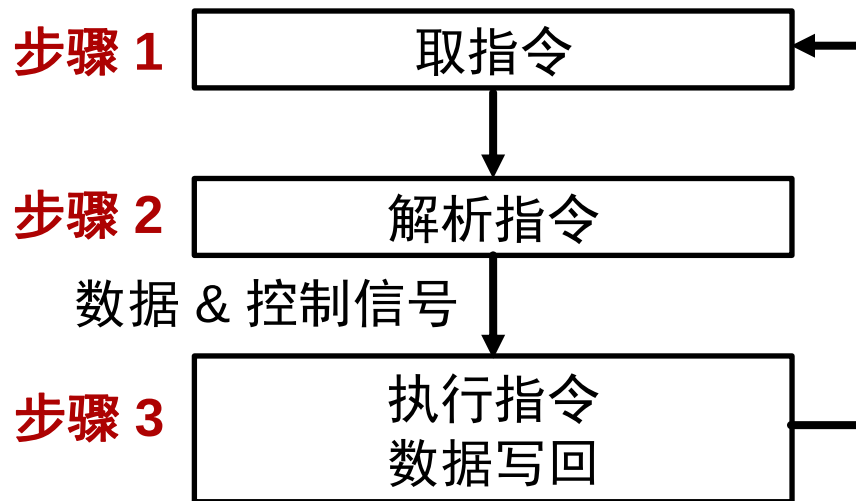
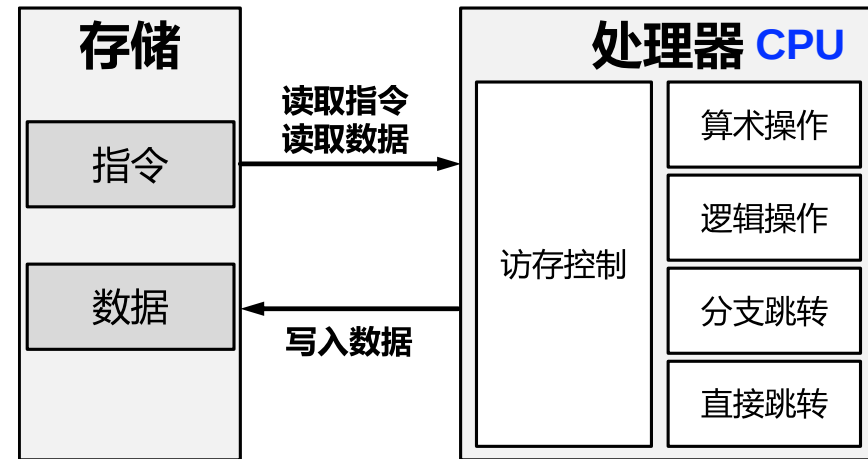
如何支持通用算法

- 支持这些基本操作！



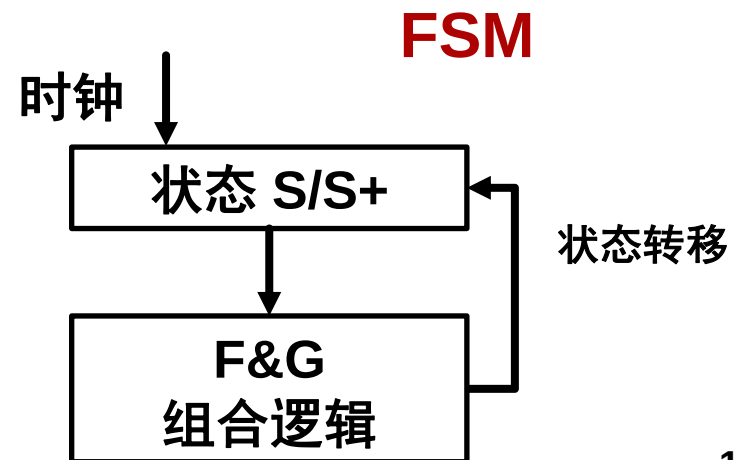
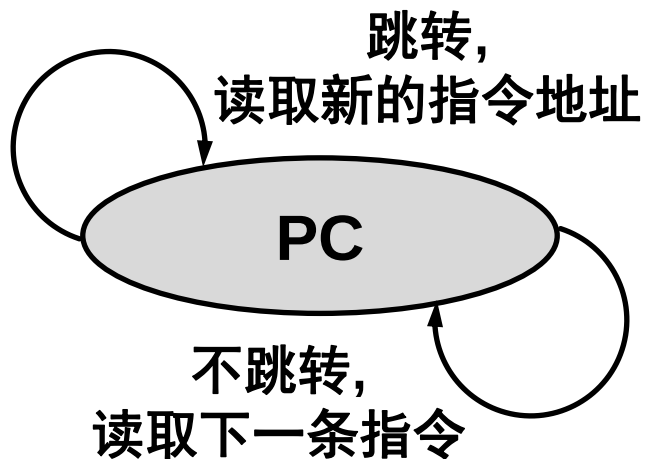
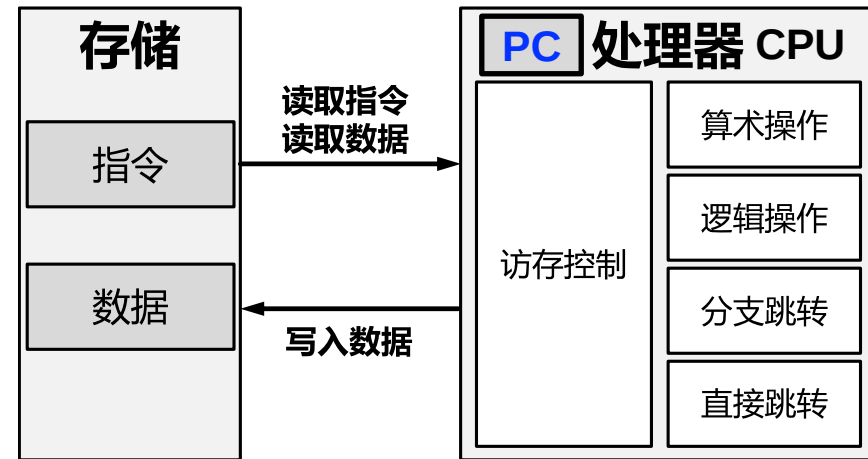
如何支持通用算法

- 支持这些基本操作！
- 如何实现？
 - 构造有限状态机
 - 中央处理器：CPU



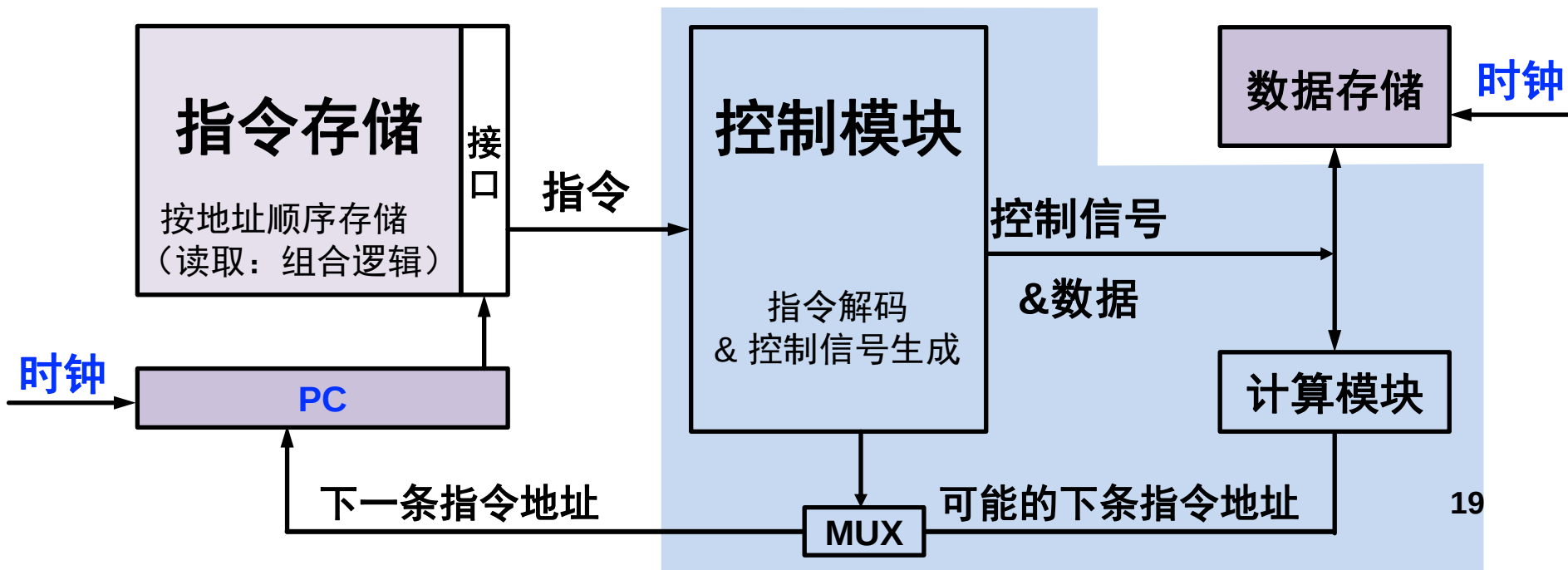
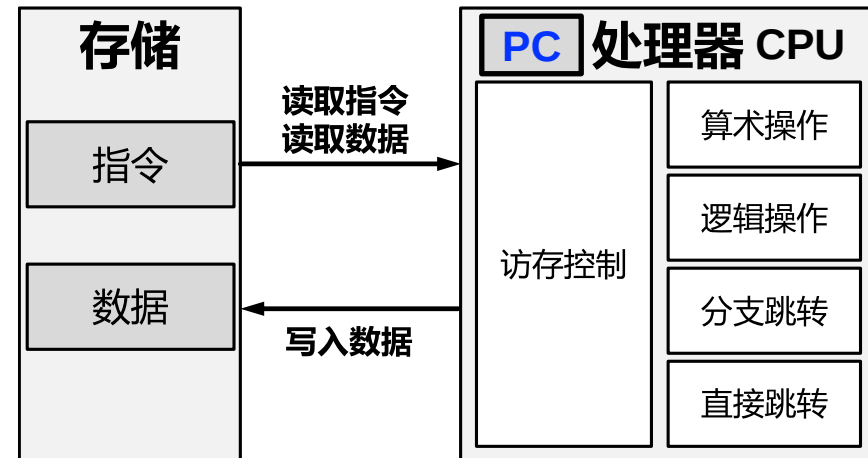
如何支持通用算法

- 程序计数器(Program Counter)
 - **PC**, 指示当前的指令地址
 - PC每个周期被更新:
 $PC \leq PC + 1^*$, or $PC \leq PC_jump$



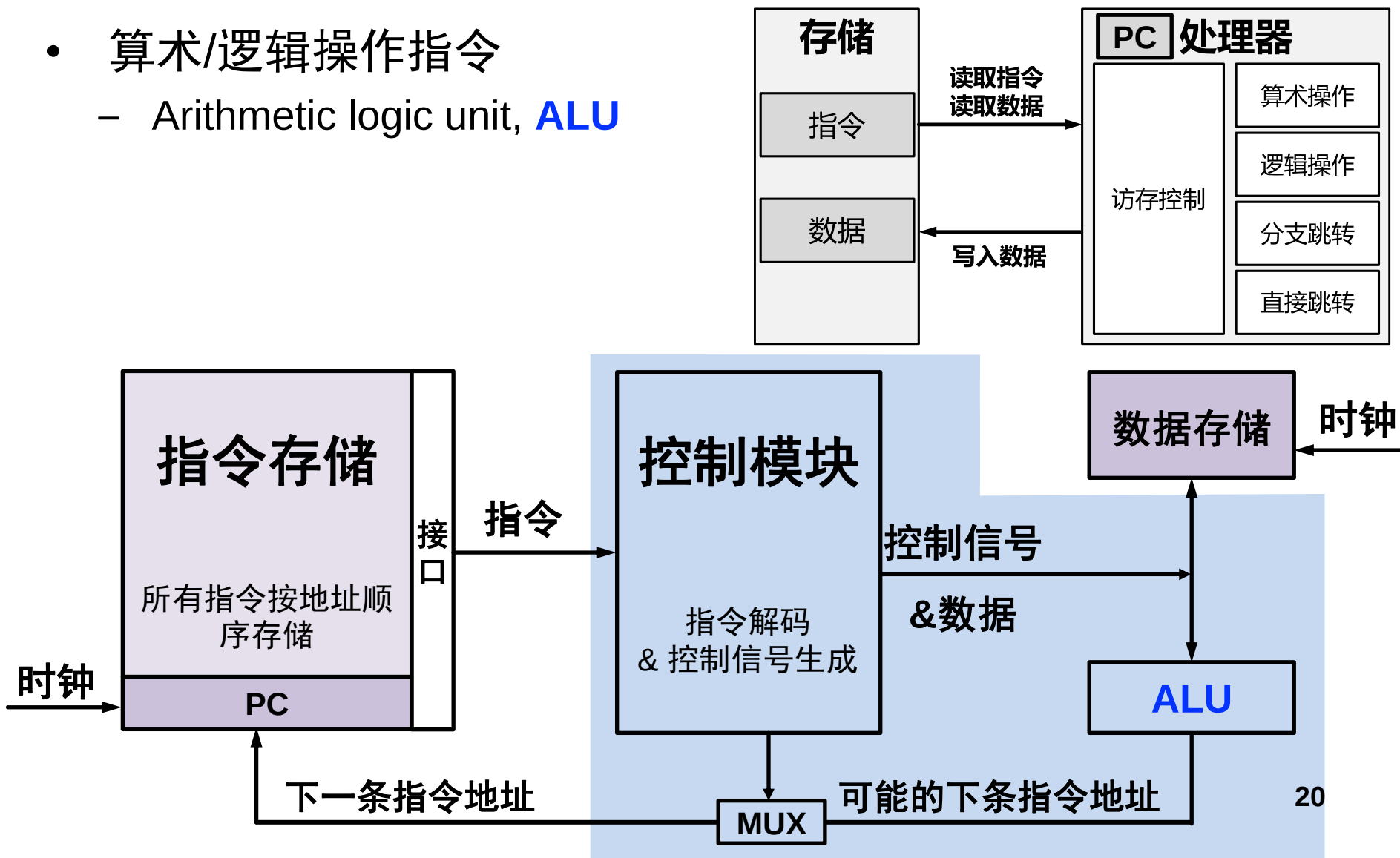
如何支持通用算法

- 程序计数器(Program Counter)
 - PC, 指示当前的指令地址
 - PC每个周期被更新:
 $PC \leq PC + 1^*$, or $PC \leq PC_jump$



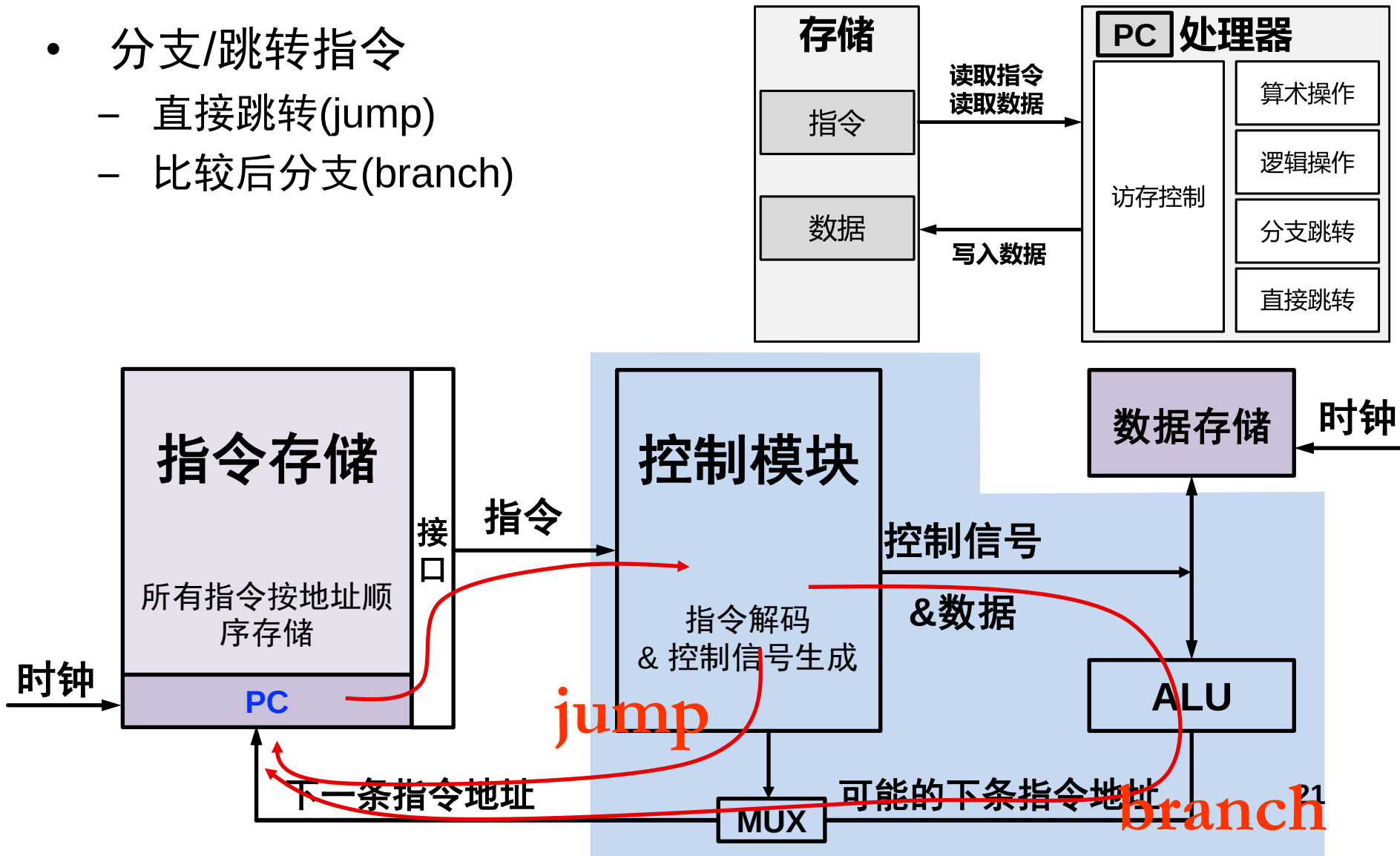
如何支持通用算法

- 算术/逻辑操作指令
 - Arithmetic logic unit, **ALU**



如何支持通用算法

- 分支/跳转指令
 - 直接跳转(jump)
 - 比较后分支(branch)



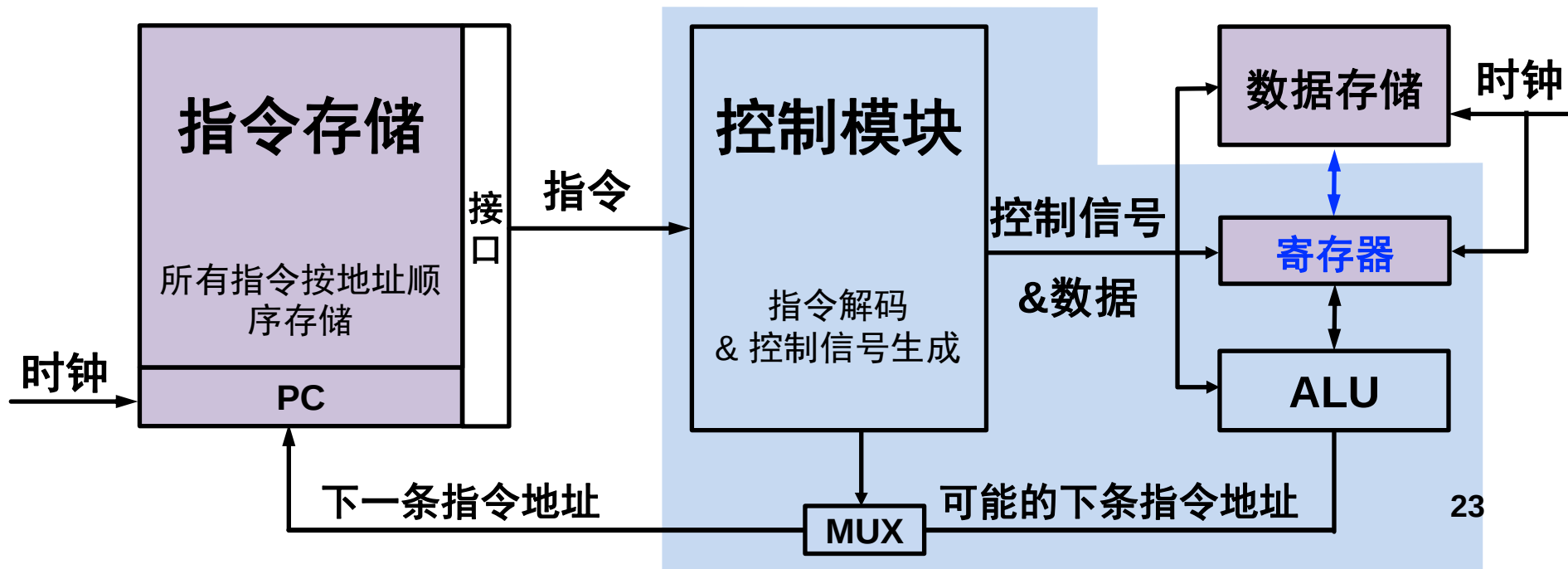
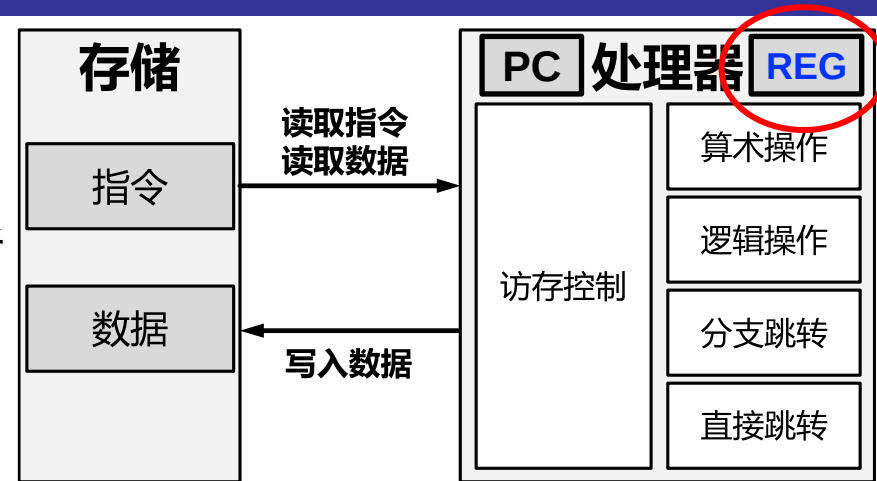


内存访问慢...

利用CPU内的高速寄存器？！

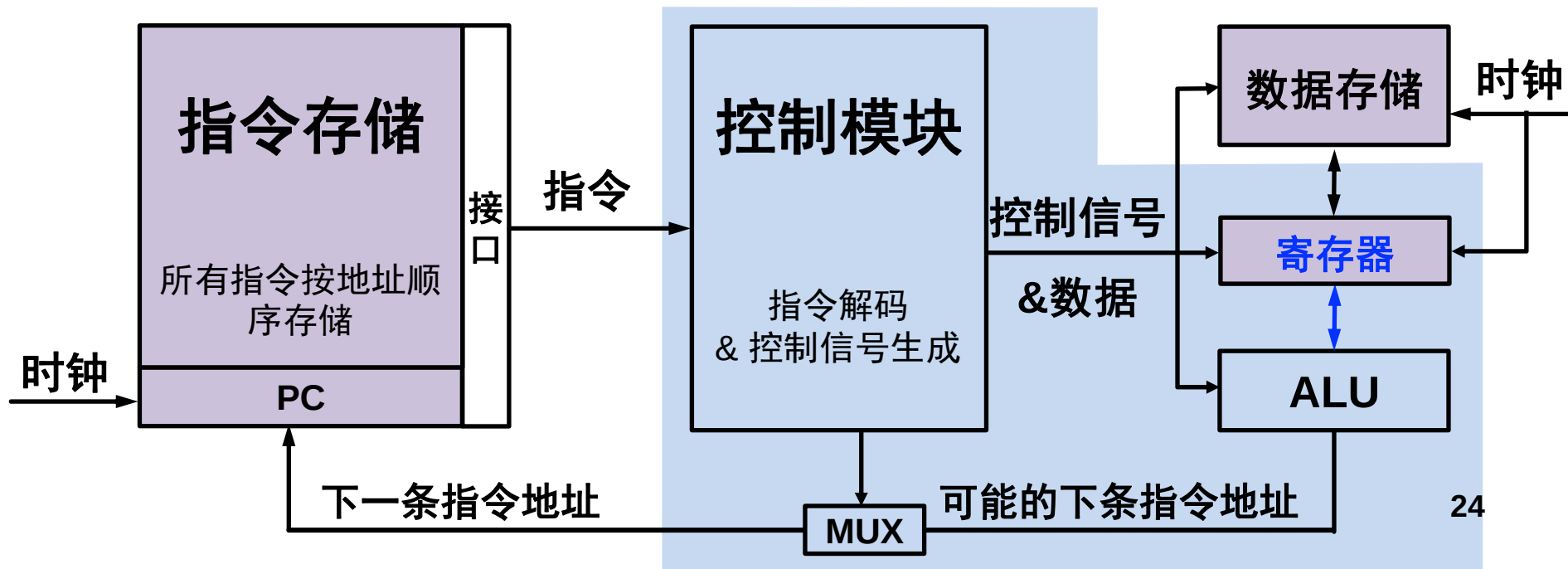
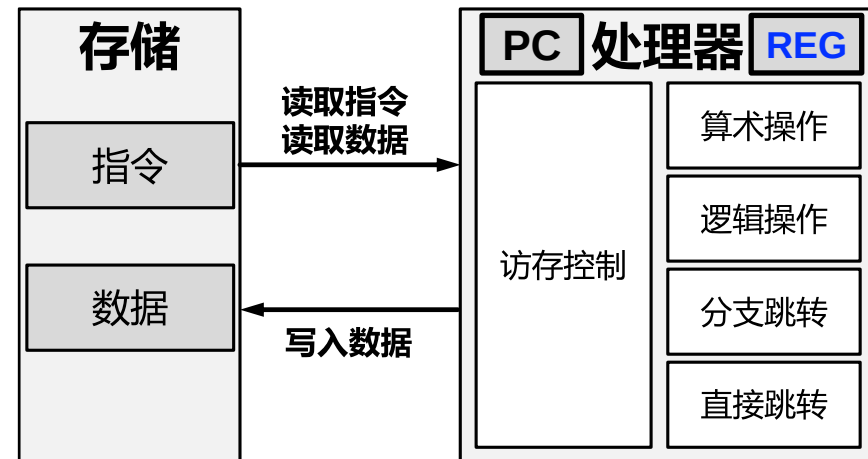
如何支持通用算法：CPU有寄存器REG?

- 存取指令
 - 装入(load)数据：数据存储在REG
 - 存储(store)数据：REG到数据存储



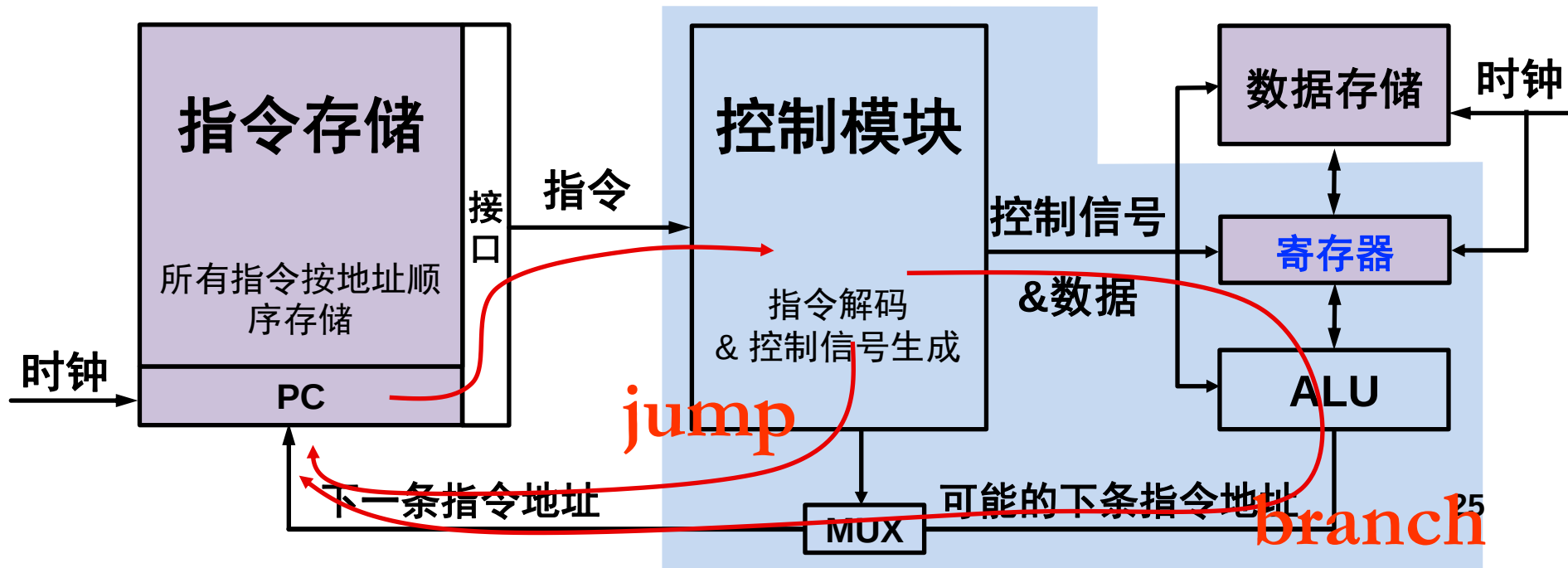
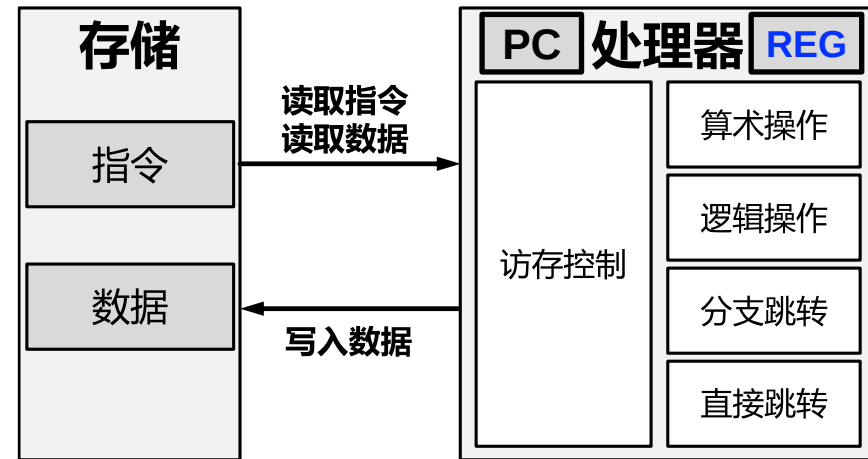
如何支持通用算法：CPU有寄存器REG？

- 算术/逻辑计算指令
 - REG/Constant到REG
 - ALU不直接访问数据存储



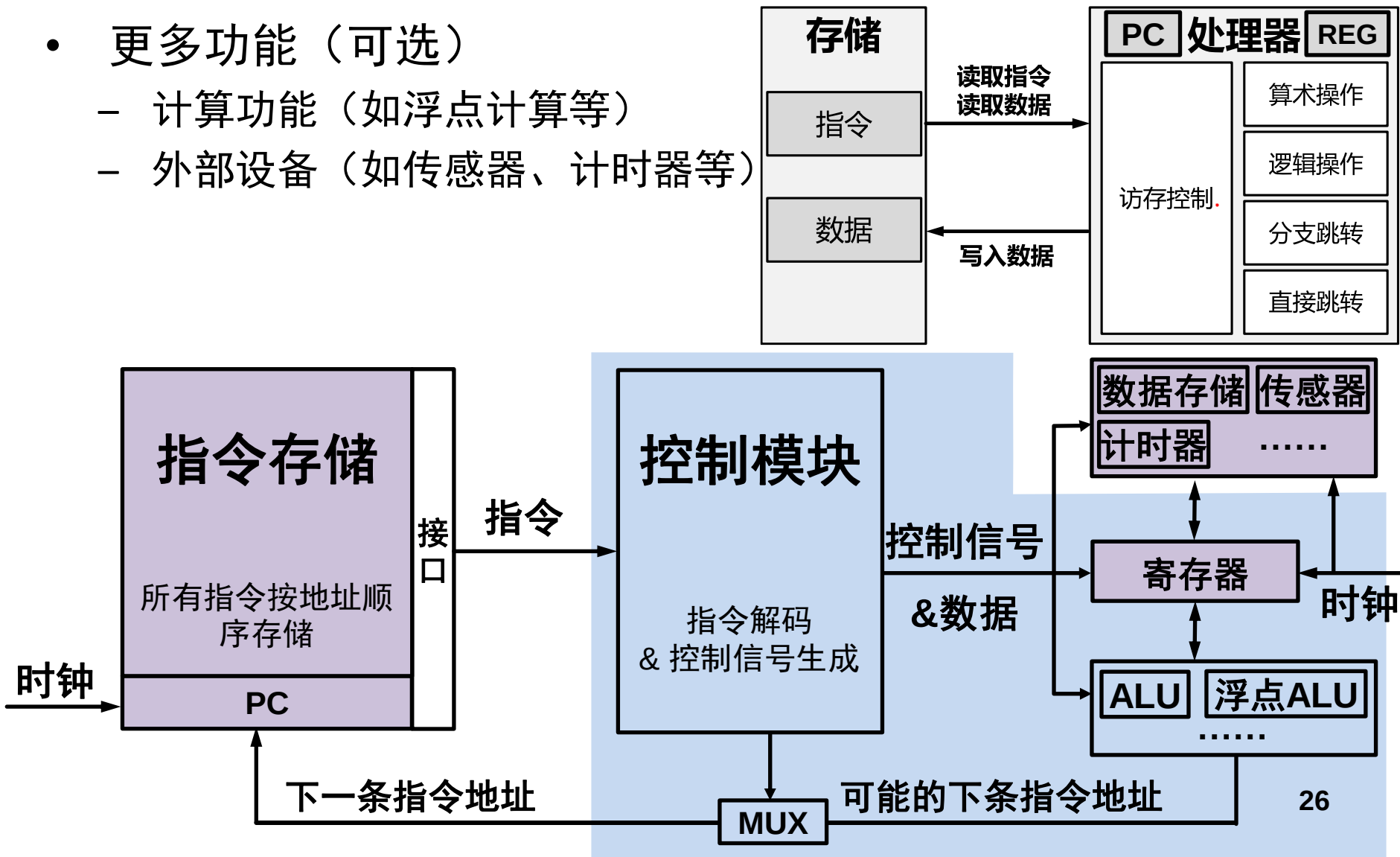
如何支持通用算法：CPU有寄存器REG？

- 分支/跳转指令
 - 直接跳转(jump)
 - 比较后分支(branch)

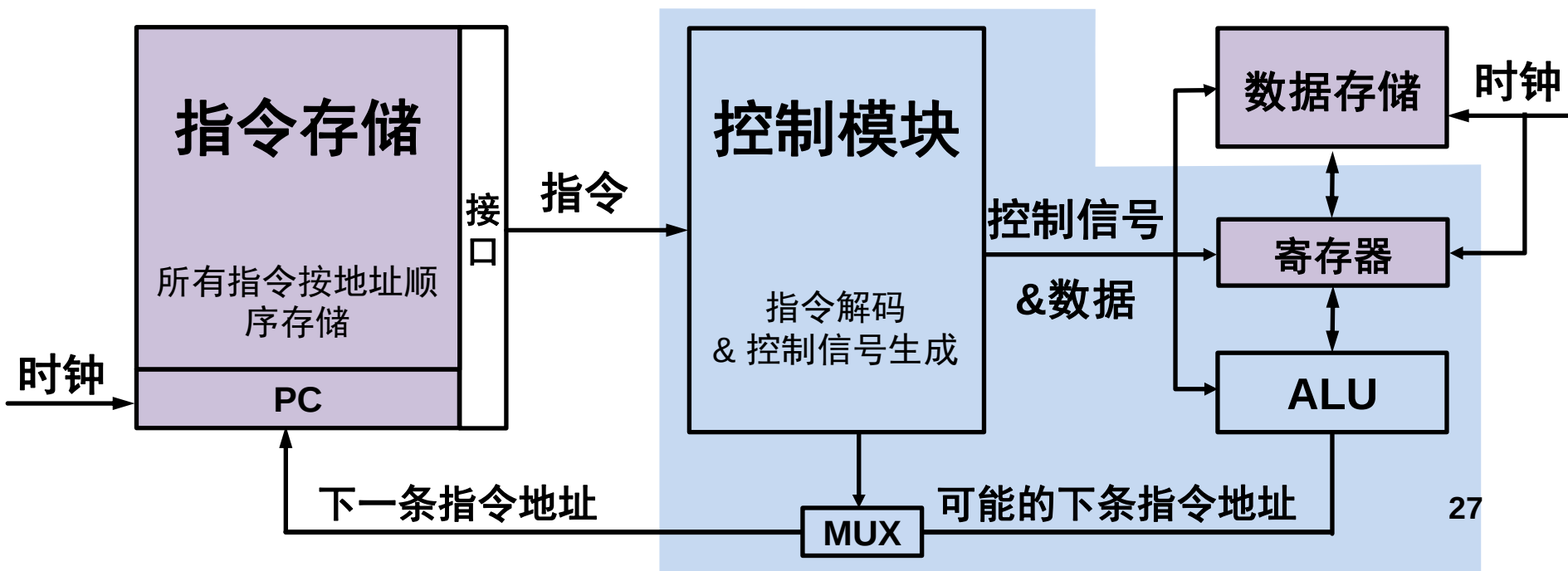
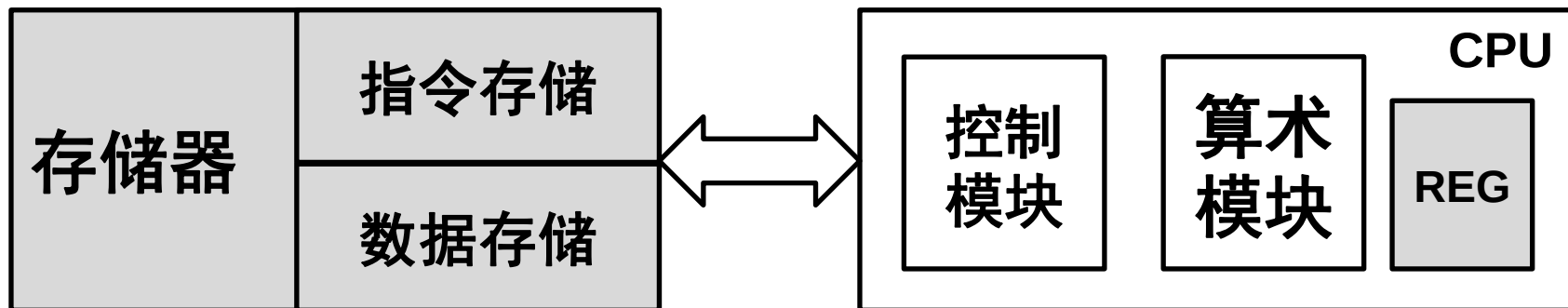


如何支持通用算法

- 更多功能（可选）
 - 计算功能（如浮点计算等）
 - 外部设备（如传感器、计时器等）



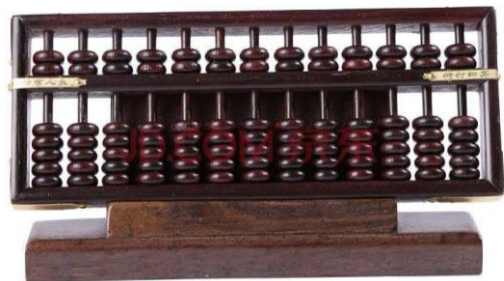
从专用芯片到处理器：总结



本节课目录

- 引言
 - 从算法到处理器
 - 通用处理器的结构
 - 程序执行流程
- 指令集设计
- 总结

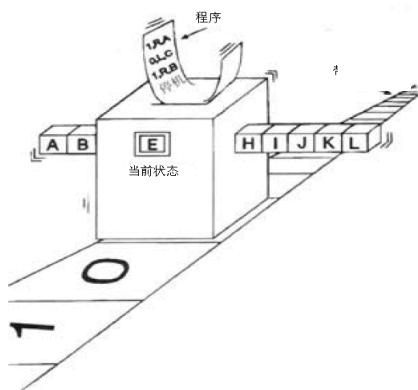
处理器的演变



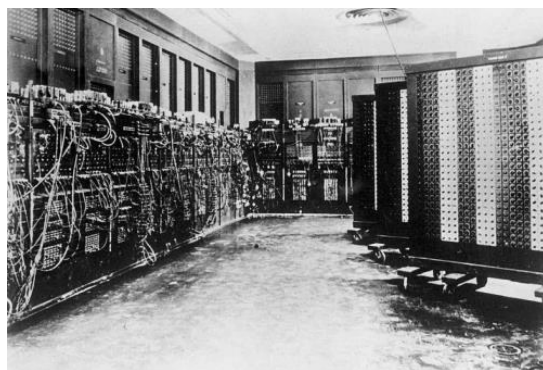
算盘
(北宋)



机械计算机
(17世纪)



图灵机模型
(1936年)



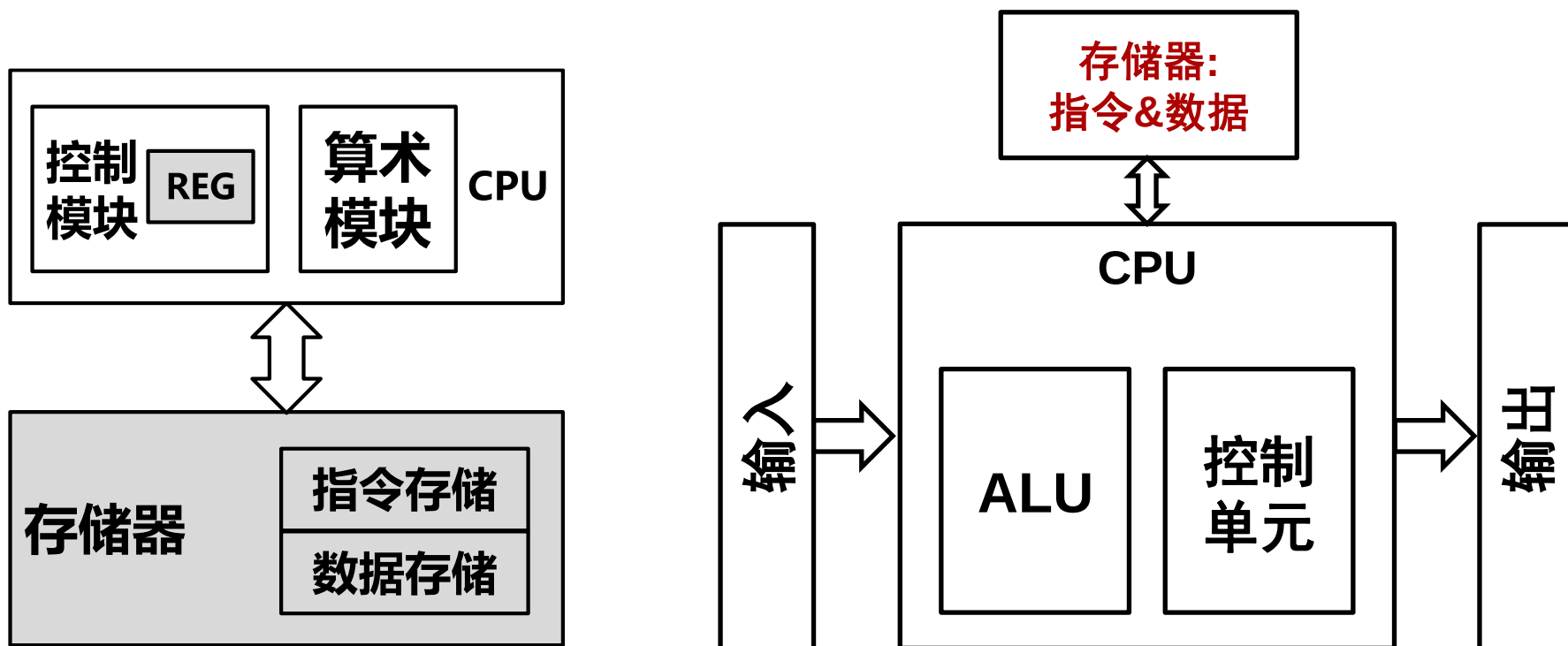
电子管计算机
(1946年)



处理器芯片
(1971年至今)

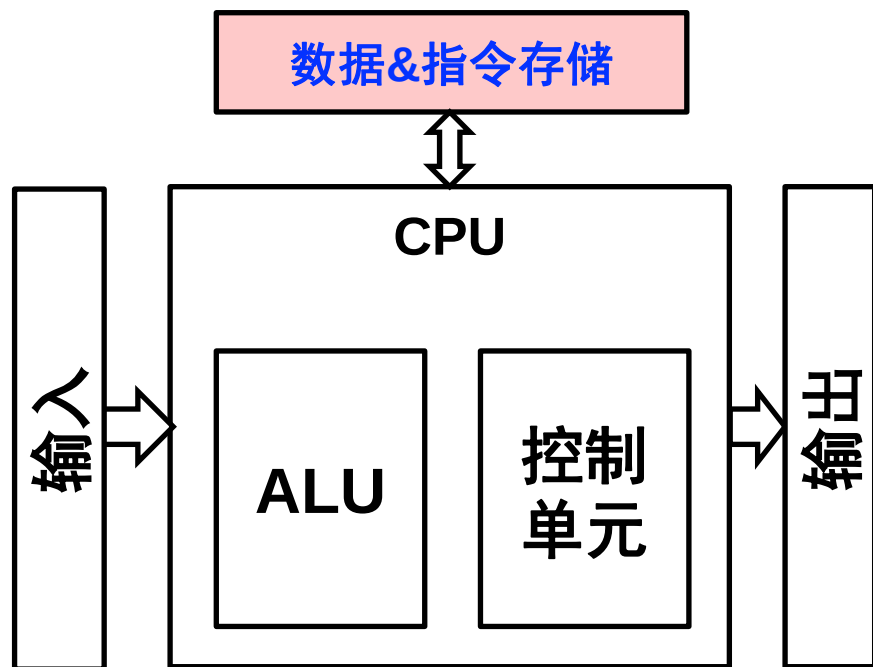
处理器的演变：普林斯顿架构

- 普林斯顿架构*（或称冯·诺依曼架构）
 - 存储器同时存储指令和其他数据
 - CPU处理器包括控制单元和计算单元（ALU）

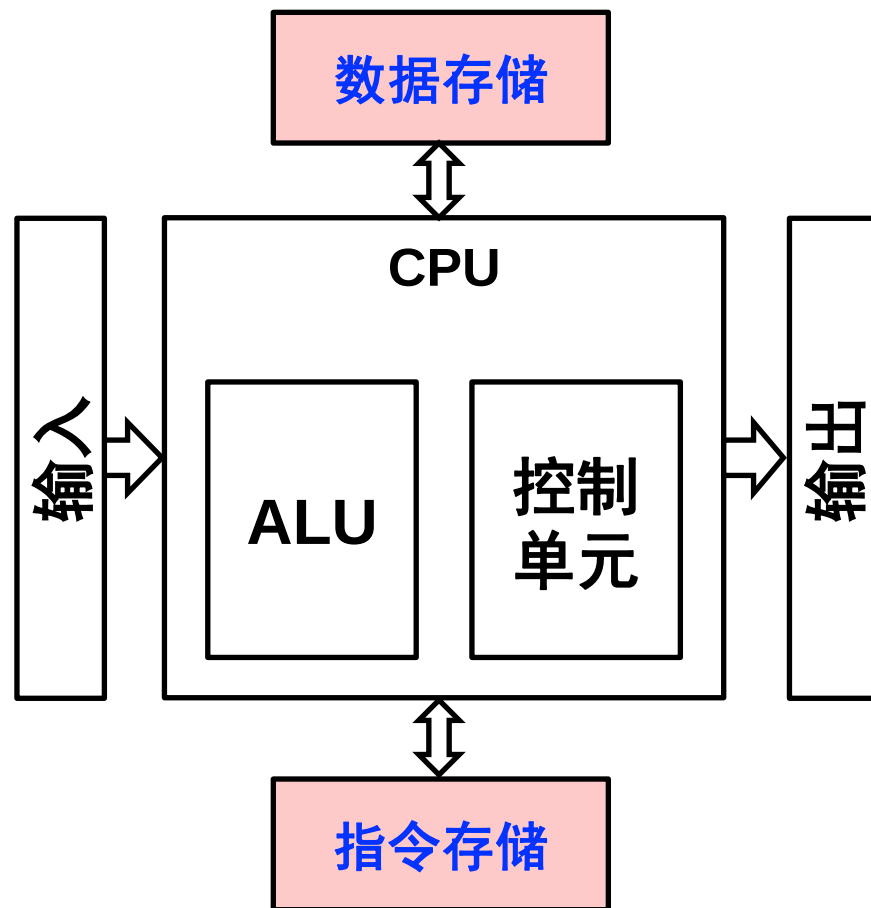


处理器的演变：哈佛架构

普林斯顿架构



哈佛架构



本节课目录

- 引言
 - 从算法到处理器
 - 通用处理器的结构
 - 程序执行流程
- 指令集设计
- 总结

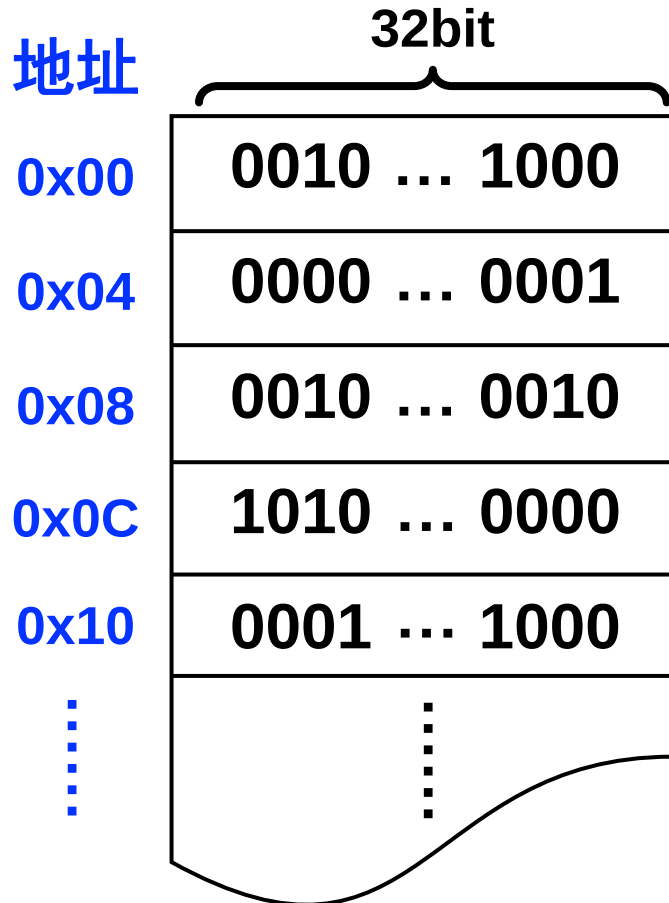
程序执行流程： 数据存储格式

- 使用0/1组成的二进制序列表示数据
- 指令和数据被编码为0和1，存放在指定地址

地址	内容
0x0	00100000 00001000 00000000 00000001
0x4	00100001 00001000 00000000 00000010
0x8	10101100 00001000 00011111 01000000
⋮	⋮

指令集：存储结构

- 指令集架构中数据应如何存储？以MIPS为例



- 数据按行存储
 - 以字节为单位索引
 - 每个地址对应1字节数据
- 字节序
 - 多字节数据的存储顺序
 - 以4字节数据0x12345678的存储为例：



程序执行流程：汇编语言

- 汇编语言
 - 使用助记符帮助阅读和编写

汇编语言源程序

```
addi $s1, $0, 1
addi $s1, $s1, 2
sw $s1, 8000($0)
```

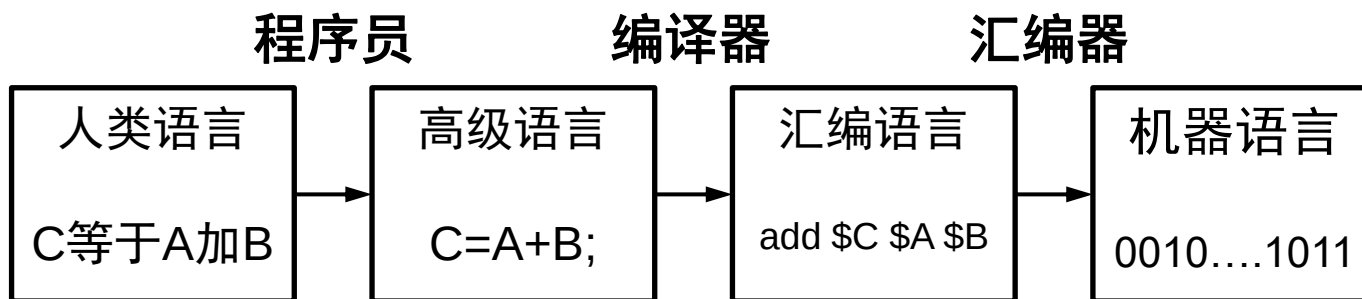
汇编器 (Assembler)
翻译

机器语言程序

```
0010...
0010...
1010...
```

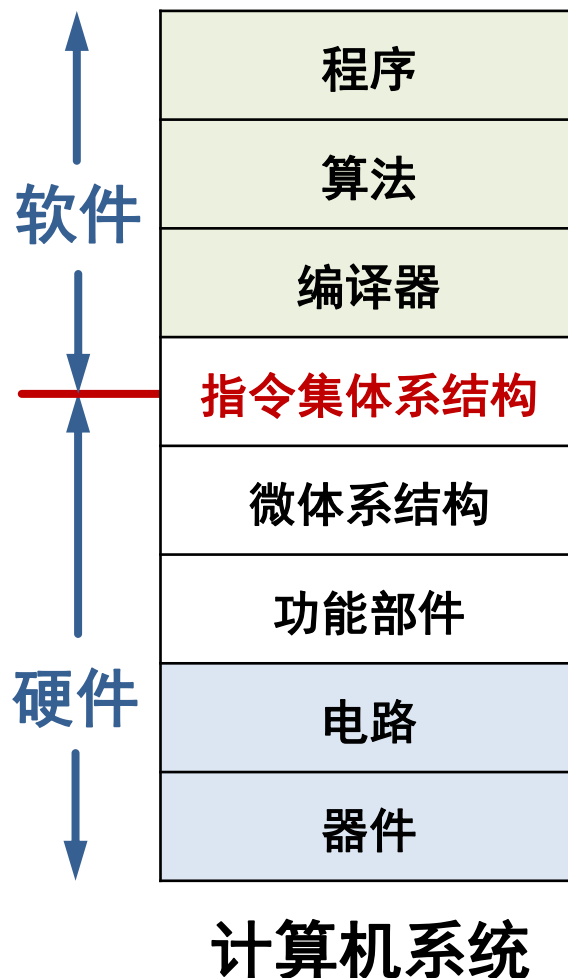
程序执行流程：高级语言

- 实现更高的开发效率
 - 高级语言：C、Java、Python等等
- 运行高级语言程序
 - 使用编译器（Compiler）**编译**
 - 高级语言 → 等价汇编指令序列 → 等价机器语言程序



- 使用解释器（Interpreter）**解释**
 - 逐条执行高级指令

引言总结



本节课目录

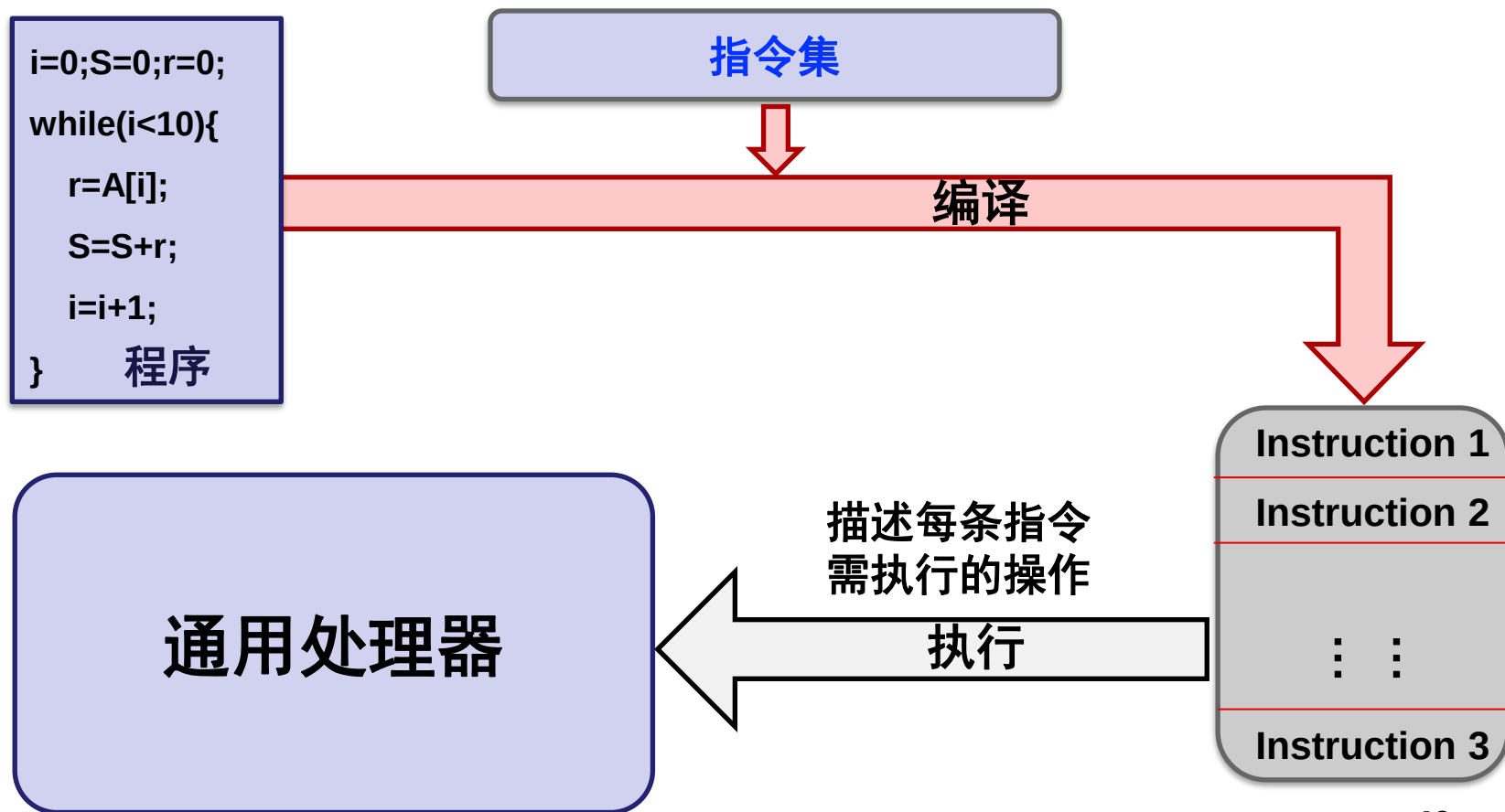
- 引言
- 指令集设计
 - 指令集：角色和地位
 - 指令集：性能评价
 - 指令集：数据访问
 - 指令集设计
- 总结

指令集：角色

- 问题：指令集设计目标？
 - 执行高效
 - 硬件复杂度低
 - 其他指标（功耗、可靠性等）

指令集：角色

- 软件和通用CPU间的桥梁



指令集：角色

- 指令集

- 程序和指令：定义？

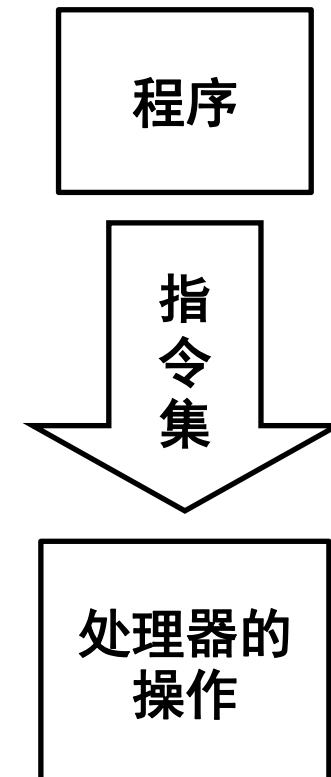
- 指令集

- 指令类型的集合
 - 控制处理器的规则表

- 指令集架构(ISA)

- 使用有限长序列完成通用任务
 - 完整指令集合

通用计算机



本节课目录

- 引言
- 指令集设计
 - 指令集：角色和地位
 - 指令集：性能评价
 - 指令集：数据访问
 - 指令集设计
- 总结

性能评价

- 计算机性能评价

$$\text{性能} = \frac{1}{\text{执行时间}}$$

— 其他评价标准：面积、功耗等



性能评价

- CPU执行任务的时间

- 总时间 = 等待I/O的时间 + 执行时间

- 执行时间 = 时钟周期数 $\times$ 时钟周期 $T = \frac{\text{时钟周期数}}{\text{时钟频率 } f}$

- 时钟周期数 = 程序指令数 $\times$ 每条指令平均周期数

- 每条指令平均周期数

- 又称为 **CPI**: Cycles Per Instruction

执行时间 =
指令数 $\times$ CPI $\times$ 时钟周期

性能评价

- **CPI: cycle per instruction**
 - 简单的指令需要较少的时钟周期
 - P_i —— 第*i*类指令出现的频度
 - CPI_i —— 执行第*i*类指令所需的时钟周期

$$CPI = \sum_{i=1}^n CPI_i \times P_i$$

性能评价：例子

- ISA包括三种指令：A、B和C
 - 问题1：CPI?
 - 问题2：如果可以将某个指令加速一倍（执行周期变为一半），应该加速哪个指令？

	A	B	C
执行所需周期	2	4	8
指令占比	50%	40%	10%

$$\begin{aligned}\text{CPI} &= 2*50\%+4*40\%+8*10\% \\ &= 1+1.6+0.8 = 3.4\end{aligned}$$

性能评价：例子

- ISA包括三种指令：A、B和C
 - 问题1：CPI?
 - 问题2：如果可以将某个指令加速一倍（执行周期变为一半），应该加速哪个指令？

	A	B	C
执行所需周期	2	4	8
指令占比	50%	40%	10%

$$\begin{aligned}\text{CPI} &= 2*50\% + 2*40\% + 8*10\% \\ &= 1 + 0.8 + 0.8 = 2.6\end{aligned}$$

性能评价：例子

关注瓶颈！

影响性能的因素

$$\text{执行时间} = \text{指令数} \times \text{CPI} \times \text{时钟周期}$$

- **指令数**

- 指令集架构（ISA），与指令集的具体实现无关
- 编译器

- **CPI**

- 存储系统结构、处理器结构
- 测试程序中各类指令的组成
- 编译器

- **时钟周期**

- 机器的实现细节
- 存储系统结构、处理器结构

影响性能的因素

$$\text{执行时间} = \text{指令数} \times \text{CPI} \times \text{时钟周期}$$

影响因素	影响
算法	指令数、CPI
程序设计语言	指令数、CPI
编译器	指令数、CPI
指令集体系架构	指令数、CPI、时钟周期
硬件实现	CPI、时钟周期

相互影响

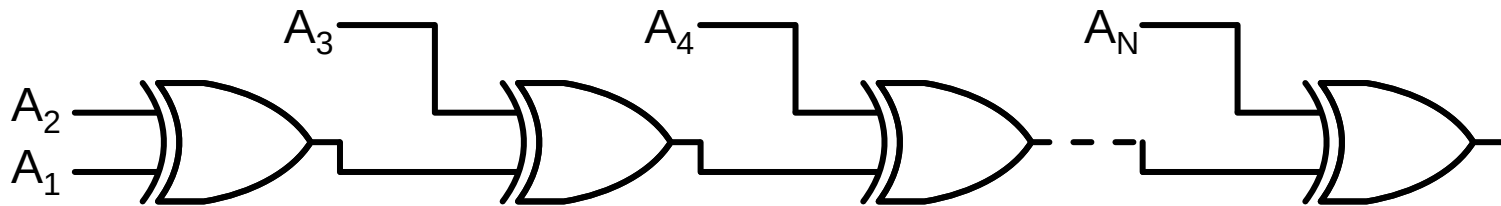
影响性能的因素

- 如何提升计算机性能？
 - 一些技术可以在保持其他两项不变的前提下减少其中的一项，例如？
 - 采用优化编译技术
 - ✓消除冗余代码，减少指令数
 - ✓不改变CPI和时钟周期
 - 采用快速电路技术或更为先进的电路结构
 - ✓减少信号传输延迟，减少时钟周期
 - ✓不改变CPI和指令数

影响性能的因素

快速电路技术：速度和规模提高了

N输入异或门设计



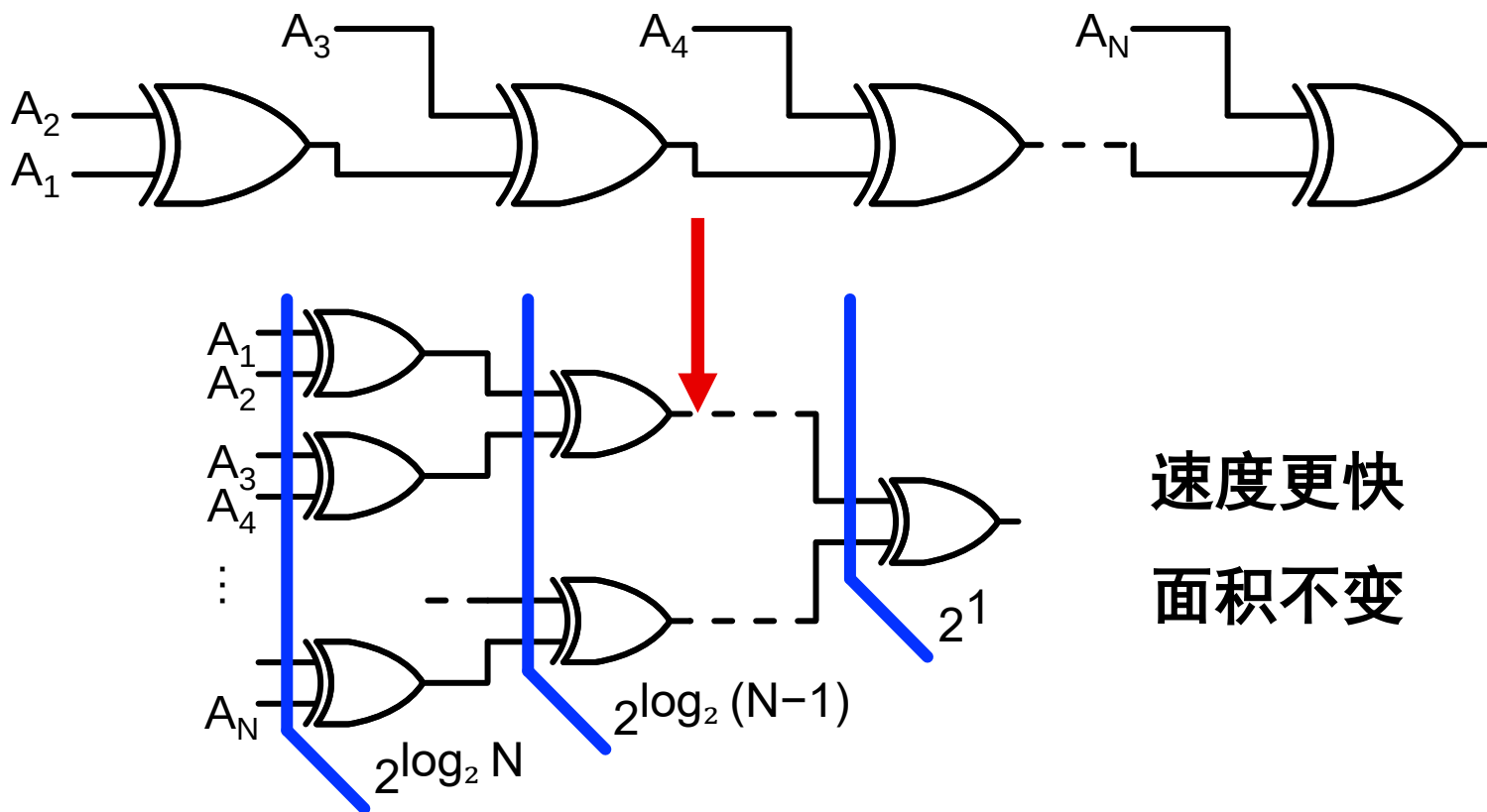
简单，但是延时高

A1输入到输出的延时是单个门延时的N倍

影响性能的因素

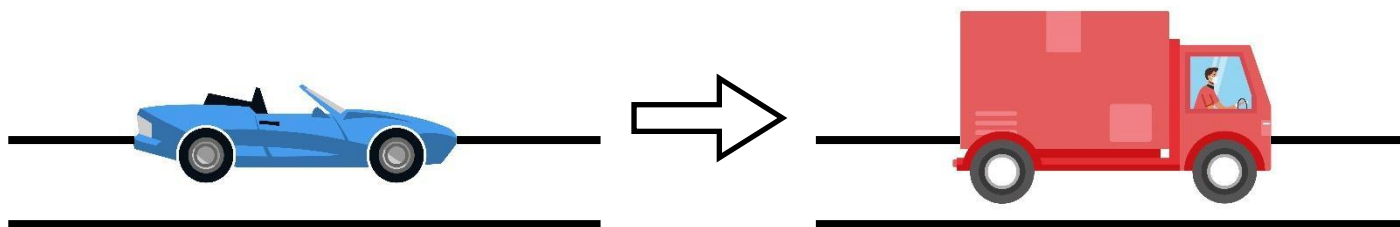
快速电路技术：速度和规模提高了

N输入异或门设计



影响性能的因素

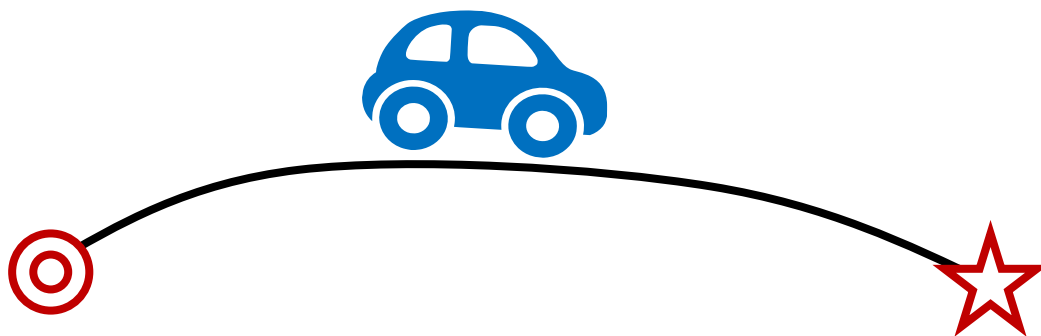
- 提高计算机性能的途径
 - 一些技术可能在减少某一项的同时，会恶化另一项或两项
 - 每条指令执行的操作数量 $\uparrow \rightarrow$ 指令数 $\downarrow$



容量更大，时间更长

影响性能的因素

- 高速公路带来的启发

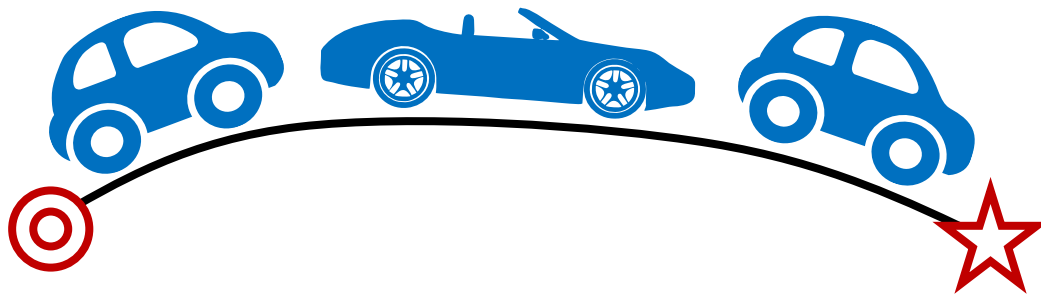


一次执行一条指令

- 一条路一辆车
- 效率低下

影响性能的因素

- 高速公路带来的启发

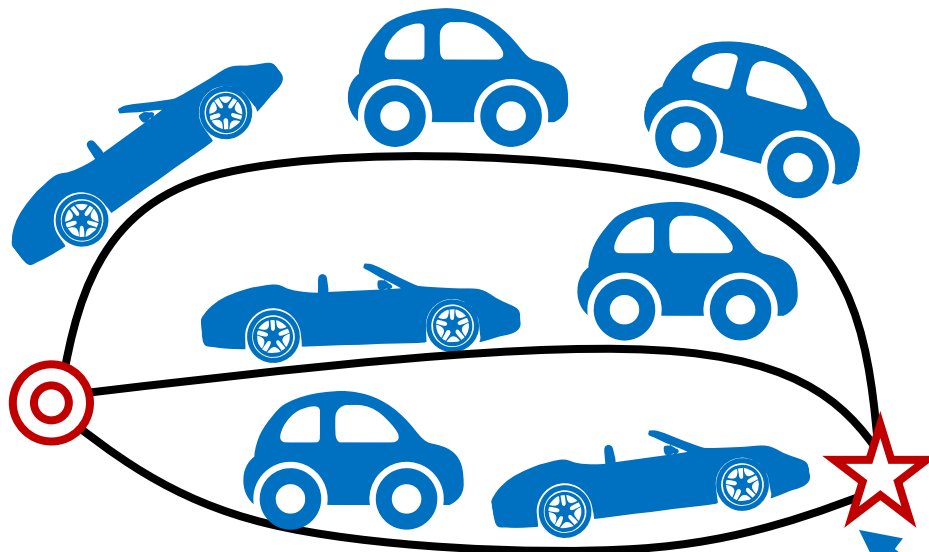


流水线执行指令

- 一条路多辆车
- 优点：效率提高
- 不同车速度不同？
- 依赖性？

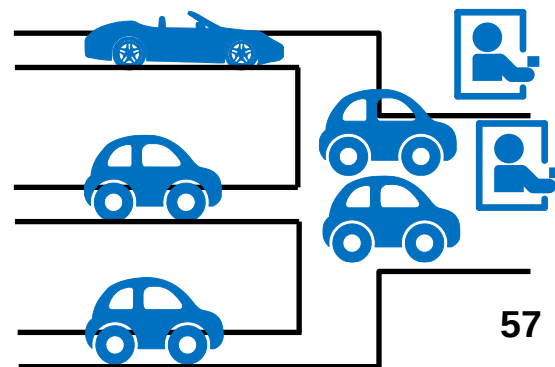
影响性能的因素

- 高速公路带来的启发



并行执行多条指令

- 多条路多辆车
- 优点：更高的吞吐量
- 现在的瓶颈？



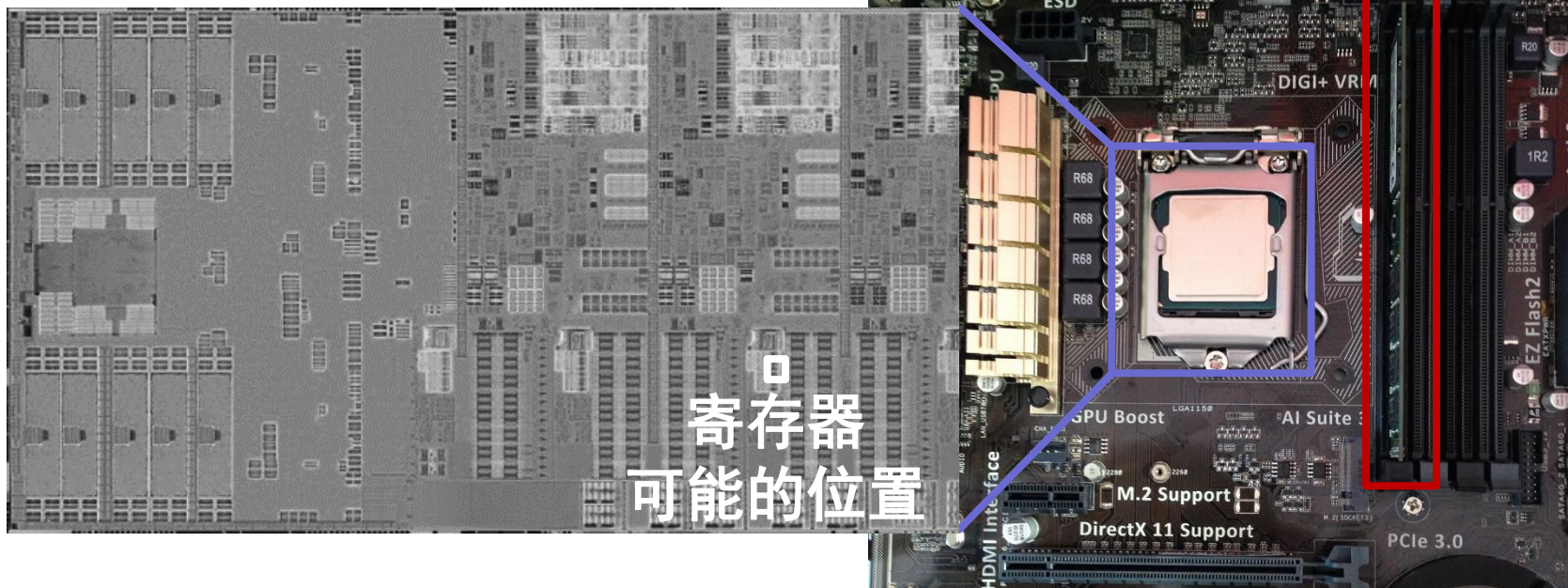
本节课目录

- 引言
- 指令集设计
 - 指令集：角色和地位
 - 指令集：性能评价
 - 指令集：数据访问
 - 指令集设计
- 总结

指令集：数据访问

- 现代处理器中的数据存储

Intel Core i7-4790K

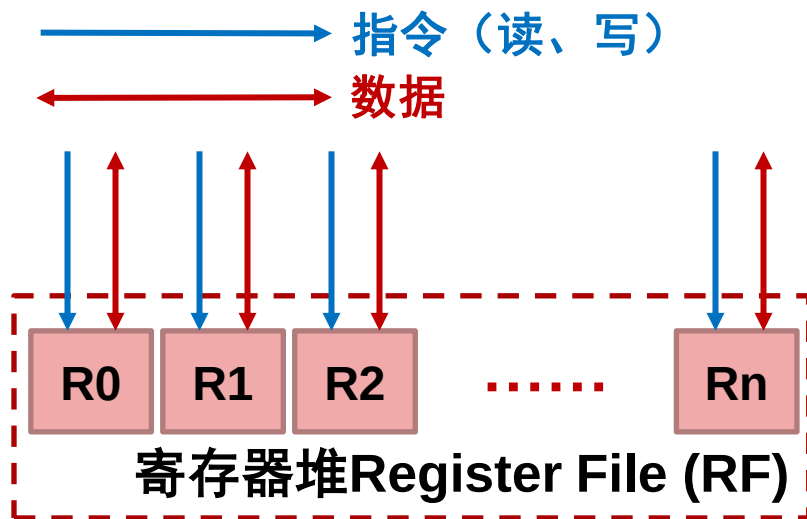


距离相差两个数量级
距离长 → 访问慢

指令集：寄存器 vs. 存储器

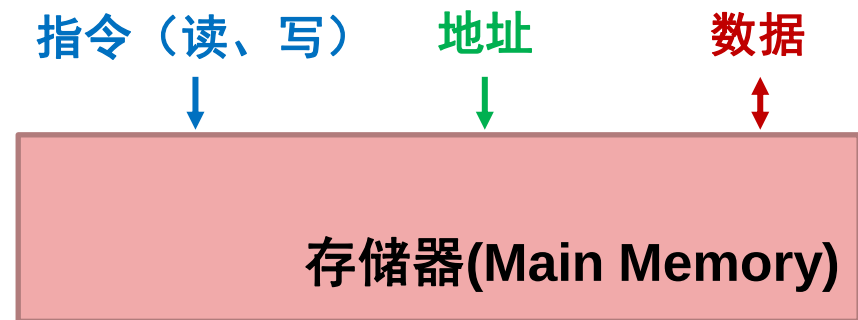
寄存器与寄存器堆

- 访问方式
 - 以**编号**进行访问
 - **可同时访问**不同寄存器
- 规模与速度
 - **个数少**，访问**速度快**



主存储器(Main Memory)

- 访问方式
 - 以**地址**进行访问
 - **不可同时访问**不同地址
- 规模与速度
 - **地址范围大**，访问**速度慢**
- 中文有时简称“**内存**”
 - 注意：乃CPU芯片“**片外存储器**”



指令集：寄存器 vs. 存储器

- 指令集架构中数据应如何存储？以MIPS为例

名称	实例	注释
主要寄存器 32个	\$s0, ...\$s7 \$t0,...\$t7	- 快速定位数据 - ALU操作数只能为立即数或寄存器数
内存(Main Memory) 2^{32} 个	字节[1] ... 字节[2^{32}]	- 只能使用数据传送指令访问 - 以字(word)、半字(half word)、字节(byte) 为单位访问 - 相邻字(Word)数据的地址相差4

MIPS中, 1 word = 4 bytes = 32 bits

回忆：数据在存储器中是如何存放的？

本节课目录

- 引言
- 指令集设计
 - 指令集：角色和地位
 - 指令集：性能评价
 - 指令集：数据访问
 - 指令集设计
- 总结

指令集设计：RISC vs CISC

- 在设计指令集时应该考虑哪些因素？

精简/复杂
指令集计算机

- 指令集能够满足我们所有的需求
- 指令集的指令数目少 or 多？

精简指令集RISC

- 指令数目少
- CPU设计更加简洁

↓
时钟周期短
指令数多

复杂指令集CISC

- 指令种类多
- 完成任务需要的指令少

↓
时钟周期长
指令数少

$$\text{执行时间} = \text{指令数} \times \text{CPI} \times \text{时钟周期}$$

指令集设计：RISC vs CISC

- 例：计算 $1+2+3+\dots+100$

RISC（例：MIPS）

Sum:

```
addi $t0, $zero, 1;
addi $s0, $zero, 100;
addi $v0, $zero, 0;
```

For:

```
bgt $t0, $s0, Exit;
add $v0, $v0, $t0;
addi $t0, $t0, 1;
j For;
```

Exit:

```
# Exit
```

三条指令实现for循环

CISC（例：x86）

Sum:

```
mov ecx, 100;
mov eax, 0;
```

For:

```
add eax, ecx;
loop For;
```

Exit:

```
# Exit
```

一条指令实现for循环

指令集设计：RISC vs CISC

- 复杂指令集（CISC, complex instruction set computing）
 - 指令种类多
 - 每条指令字长不等
 - 完成相同任务需要的指令数目少

代表：x86
- 精简指令集（RISC, reduced instruction set computing）
 - 指令种类少
 - 每条指令字长相等
 - 完成相同任务需要的指令数目多

代表：MIPS, RISC-V
- 最简指令集（URISC, ultimate reduced instruction set computer）
 - 只有一条指令
 - 完备的指令集
 - 一般用于教学和科研

指令集设计：示例

- 考虑一个实际的计算程序的需求

```
#define N 10

int main()
{
    int i, s[N];
    for(i = 0; i < N; ++i)
    {
        s[i] = Fib(i);
    }
    return 0;
}
```

跳转 (red arrow pointing from the end of the for loop to the start)

过程调用 (red oval around `s[i] = Fib(i);`)

访存 (red oval around `s[i]`)

```
int Fib(int n)
{
    int i, nextTerm, t0, t1;
    t0 = 0;
    t1 = 1;
    for(i = 1; i <= n; ++i)
    {
        nextTerm = t0 + t1;
        t0 = t1;
        t1 = nextTerm;
    }
    return t1;
}
```

赋值 (red oval around `t0 = 0;` and `t1 = 1;`)

分支 (red oval around `i <= n;`)

算术操作 (red oval around `nextTerm = t0 + t1;`)

过程调用 (返回) (red oval around `return t1;`)

- 回顾：需要CPU FSM支持哪些操作？

指令集设计：示例

- 算术操作指令：变量和变量、立即数和变量之间的加、减、位运算

指令	功能	指令	功能
<i>add a0, a1, a2;</i>	$a0 = a1 + a2$	<i>addi a0, a1, 5;</i>	$a0 = a1 + 5$
<i>sub a0, a1, a2;</i>	$a0 = a1 - a2$	<i>subi a0, a1, 5;</i>	$a0 = a1 - 5$
<i>and a0, a1, a2;</i>	$a0 = a1 \& a2$	<i>andi a0, a1, 5;</i>	$a0 = a1 \& 5$
<i>or a0, a1, a2;</i>	$a0 = a1 a2$	<i>ori a0, a1, 5;</i>	$a0 = a1 5$
<i>xor a0, a1, a2;</i>	$a0 = a1 \wedge a2$	<i>xori a0, a1, 5;</i>	$a0 = a1 \wedge 5$
<i>sll a0, a1, a2;</i>	$a0 = a1 \ll a2$	<i>slli a0, a1, 5;</i>	$a0 = a1 \ll 5$
<i>srl a0, a1, a2;</i>	$a0 = a1 \gg a2$ (逻辑移位)	<i>srl i a0, a1, 5;</i>	$a0 = a1 \gg 5$ (逻辑移位)
<i>sra a0, a1, a2;</i>	$a0 = a1 \gg a2$ (算术移位)	<i>srai a0, a1, 5;</i>	$a0 = a1 \gg 5$ (算术移位)

指令集设计：示例

- 分支指令

- 比较变量和变量、变量和立即数之间的大小并进行跳转

指令	功能	指令	功能
<i>beq a0, a1, label;</i>	如果 $a0 == a1$, 则跳转到 <i>label</i>	<i>beqi a0, 5, label;</i>	如果 $a0 == 5$, 则跳转到 <i>label</i>
<i>blt a0, a1, label;</i>	如果 $a0 < a1$, 则跳转到 <i>label</i>	<i>blti a0, 5, label;</i>	如果 $a0 < 5$, 则跳转到 <i>label</i>
<i>bgt a0, a1, label;</i>	如果 $a0 > a1$, 则跳转到 <i>label</i>	<i>bgti a0, 5, label;</i>	如果 $a0 > 5$, 则跳转到 <i>label</i>

- 跳转指令：直接跳转到某一条指令的位置

指令	功能
<i>j label;</i>	直接跳转到 <i>label</i>

指令集设计：示例

- 赋值指令：将一个变量/常数的值赋值给另一个变量

指令	功能	指令	功能
<code>mov a0, a1;</code>	将变量 <code>a1</code> 的值赋给 <code>a0</code>	<code>movi a0, 4;</code>	将常量4的值赋给 <code>a0</code>

- 访存指令：从存储器读取/向存储器写入数据

指令	功能	指令	功能
<code>lw a0, addr;</code>	从存储器地址 <code>addr</code> 取出数据赋值给变量 <code>a0</code>	<code>sw a0, addr;</code>	将变量 <code>a0</code> 的数据写入到存储器地址 <code>addr</code>

- 过程调用指令：跳转到被调过程和返回主调过程

指令	功能	指令	功能
<code>call a0, label;</code>	将返回指令地址赋值给变量 <code>a0</code> ，然后调用 <code>label</code> 处的过程	<code>ret a0;</code>	返回到变量 <code>a0</code> 所存储的指令地址

指令集设计：示例

- 总结现在已经设计的指令

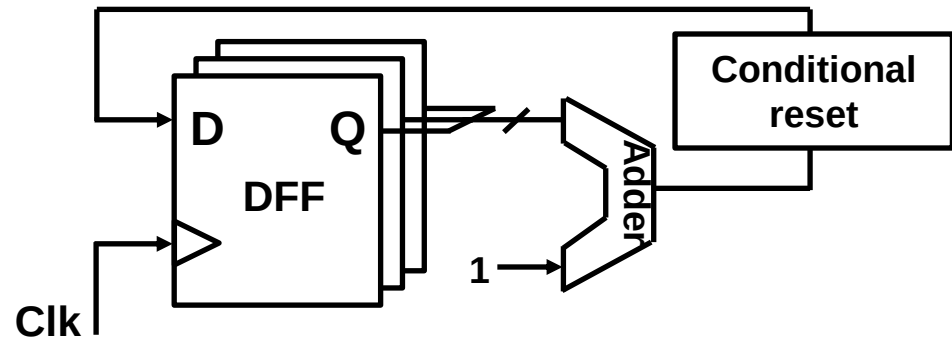
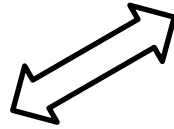
算术指令			
<i>add a0, a1, a2;</i>	<i>addi a0, a1, 5;</i>	<i>sub a0, a1, a2;</i>	<i>subi a0, a1, 5;</i>
<i>and a0, a1, a2;</i>	<i>andi a0, a1, 5;</i>	<i>or a0, a1, a2;</i>	<i>ori a0, a1, 5;</i>
<i>xor a0, a1, a2;</i>	<i>xori a0, a1, 5;</i>	<i>sll a0, a1, a2;</i>	<i>slli a0, a1, 5;</i>
<i>srl a0, a1, a2;</i>	<i>srli a0, a1, 5;</i>	<i>sra a0, a1, a2;</i>	<i>srai a0, a1, 5;</i>
分支指令			
<i>beq a0, a1, label;</i>	<i>beqi a0, 5, label;</i>	<i>blt a0, a1, label;</i>	<i>blti a0, 5, label;</i>
<i>bgt a0, a1, label;</i>	<i>bgti a0, 5, label;</i>		
跳转指令		赋值指令	
<i>j label;</i>		<i>mov a0, a1;</i>	<i>movi a0, 4;</i>
访存指令		过程调用指令	
<i>lw a0, addr;</i>	<i>sw a0, addr;</i>	<i>call a0, label;</i>	<i>ret a0;</i> 70

指令集设计：示例

- 使用我们设计的指令集实现程序

```
int cnt = 0;
for (i=0;;i++)
{
    cnt += 1;
    if(cnt > 7)
        cnt = 0;
}
```

专用硬件实现



通用CPU上的
汇编实现

计数器在通用CPU上的汇编实现

```
movi s0, 0;           # s0 = 0表示cnt
movi s1, 0;           # s1 = 0表示i
Loop:
    addi s0, s0, 1;     # s0 = s0 + 1
    bgti s0, 7, update; # 若s0 > 7则更新
    j end_update;       # 否则跳过更新
update:
    movi s0, 0;         # 更新s0 = 0
end_update:
    addi s1, s1, 1;     # s1 = s1 + 1
    j Loop;             # 继续循环
```

指令集设计：示例

- 使用我们设计的指令集实现程序

```
#define N 10

int main()
{
    int i, s[N];
    for(i = 0; i < N; ++i)
    {
        s[i] = Fib(i);
    }
    return 0;
}
```

```
main:
    movi s0, N;          # s0 = N, 表示循环次数
    movi s1, s_addr;     # s1 = s_addr表示数组起始地址
    movi a0, 0;          # a0 = 0, 用于计数
loop_outter:
    bgt a0, s0, exit;     # 如果a0 > s0则跳转到exit
    beq a0, s0, exit;     # 如果a0 == s0则跳转到exit
    call s2, fib;         # s2 = 返回地址, 然后调用fib
    slli t0, a0, 2;       # t0 = a0 << 2
    add t0, t0, s1;       # t0 = t0 + s1
    sw v0, t0;            # 将v0中的返回值存入地址t0
    addi a0, a0, 1;       # a0 = a0 + 1
    j loop_outter;        # 跳转回loop_outter
exit:
    # exit
```

Note: (1) 函数调用会将返回值默认存入v0, v1等寄存器中;
(2) 调用函数时, 要保留返回后的第一条指令地址, 这里存在s2中
→ 后续会进一步介绍调用的数值传递方法。

指令集设计：示例（续）

- 使用我们设计的指令集实现程序

```
int Fib(int n)
{
    int i, nextTerm, t0, t1;
    t0 = 0;
    t1 = 1;
    for(i = 1; i <= n; ++i)
    {
        nextTerm = t0 + t1;
        t0 = t1;
        t1 = nextTerm;
    }
    return t1;
}
```

```
# Fib函数的实现
fib:
    movi s3, 0;      # s3 = 0, 初始值0
    movi s4, 1;      # s4 = 1, 初始值1
    movi t0, 1;      # t0 = 1, 用于计数
loop_inner:
    bgt t0, a0, jump_out; # 如果t0 > a0则跳转
    add t1, s3, s4;      # t1 = s3 + s4
    mov s3, s4;          # s3 = s4
    mov s4, t1;          # s4 = t1
    addi t0, t0, 1;      # t0 = t0 + 1
    j loop_inner;        # 跳转到loop_inner
jump_out:
    mov v0, s4;          # v0 = s4
    ret s2;              # 跳转回s2存储的地址
```

指令集设计：示例

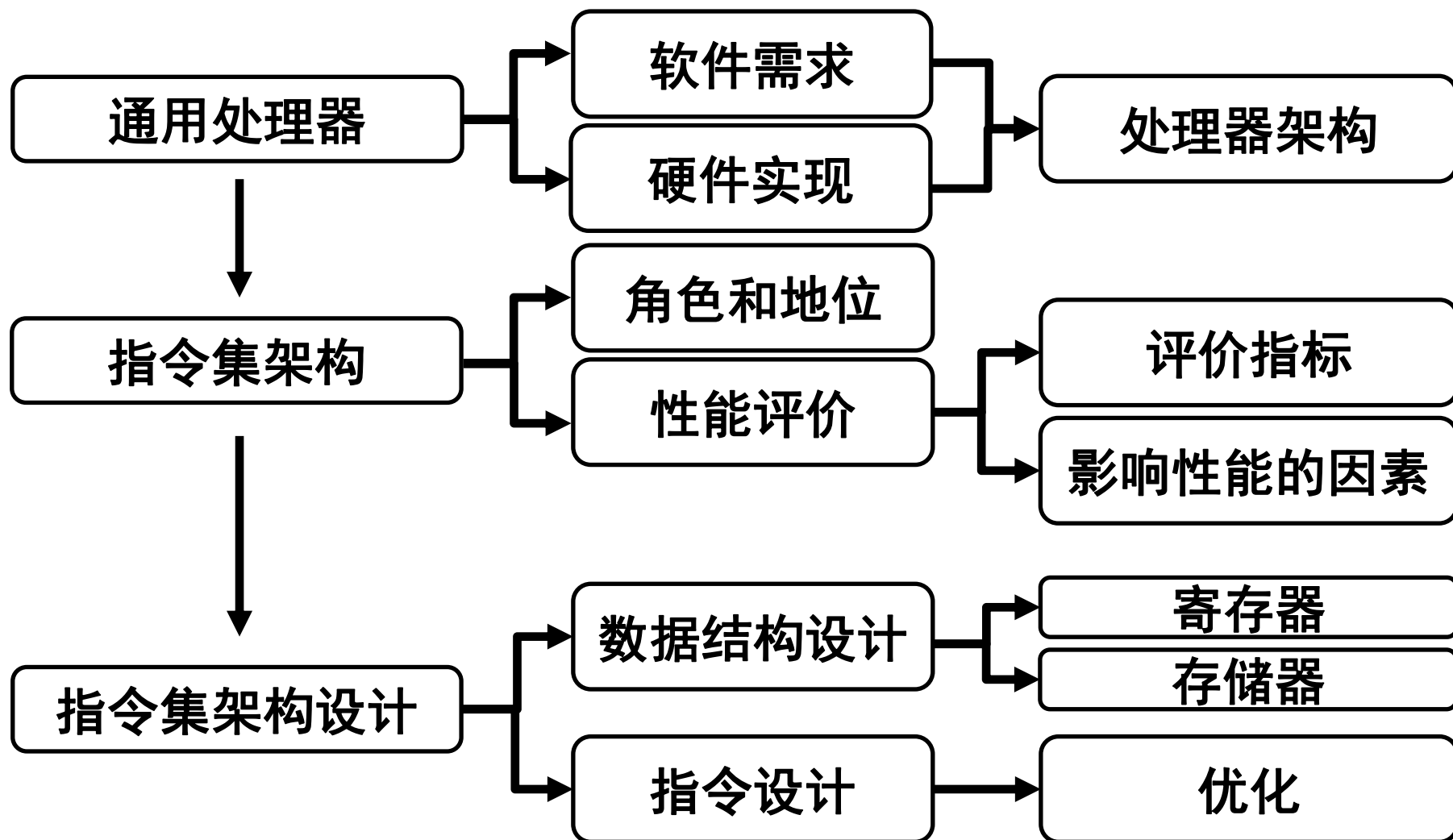
- 思考：能否精简指令集？
 - 例如：赋值指令 *mov* 和 *movi* 可以用 *addi* 指令实现

指令	替代
<i>mov a0, a1;</i>	<i>addi a0, a1, 0;</i>
<i>movi a0, 5;</i>	<i>sub a0, a0, a0;</i> <i>addi a0, a0, 5;</i>

- 优点：指令集减少了两条指令，译码与指令存储变简单
- 缺点：赋值指令需要加法器 → 执行时间变长

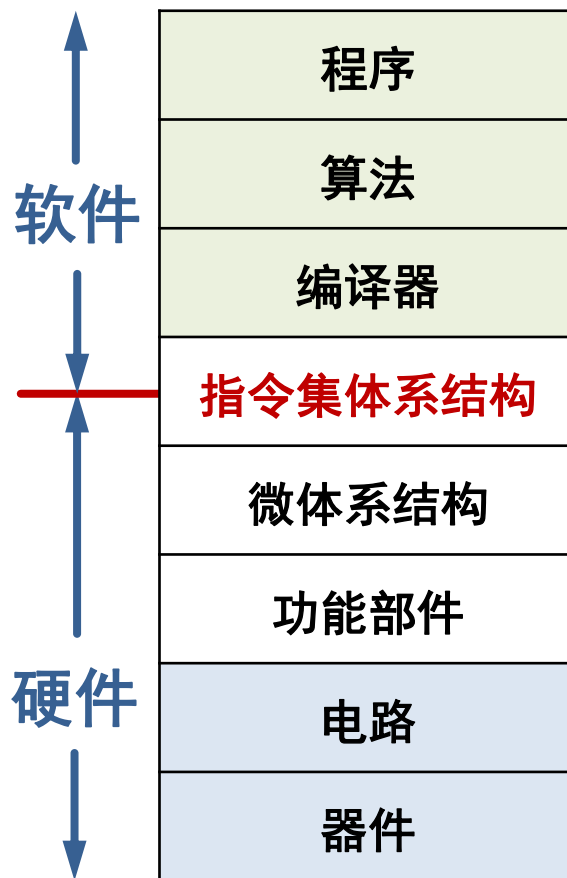
Trade-off ?

总结



总结

- 指令集架构：桥梁，需要软硬件协同设计



计算机系统

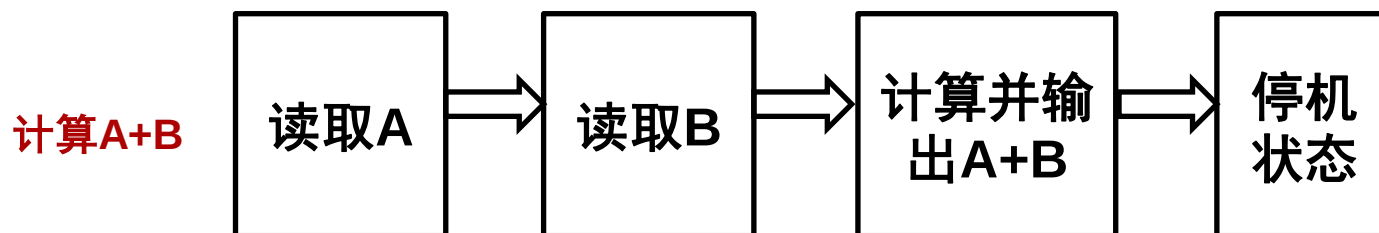
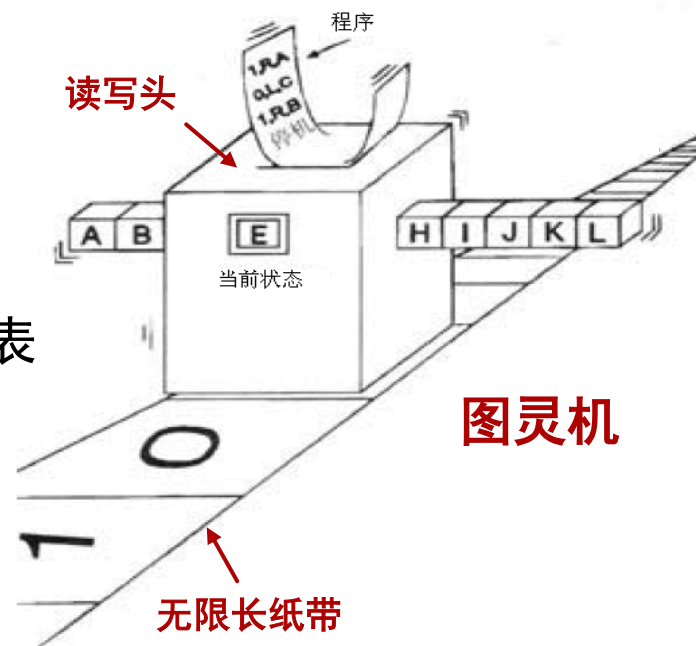
作业

- 要求
 - 线上交电子版
 - 不要抄袭，迟交一周内扣50%，答案公布后再交不给分
- 本次作业
 - 网络学堂作业区

附录：图灵机

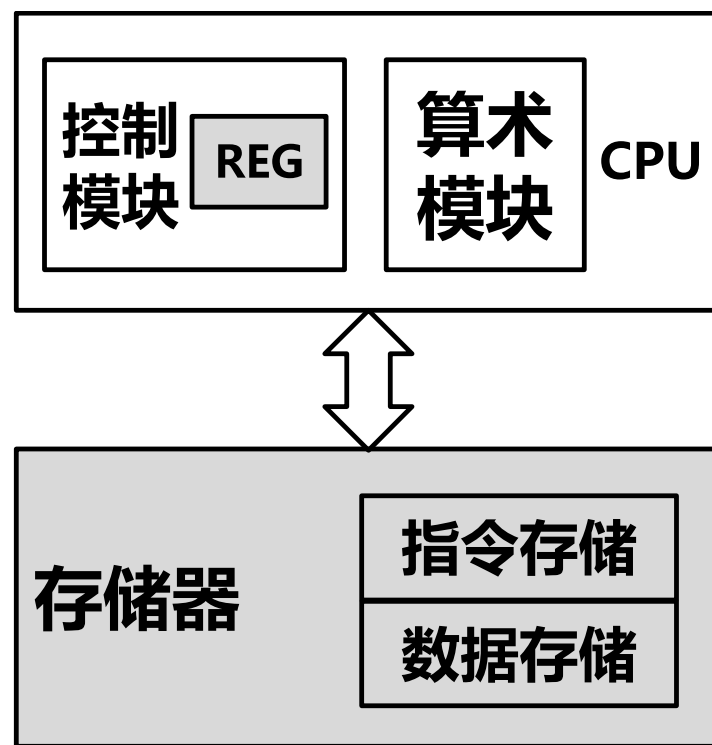
• 图灵机

- 纸带、读写头、规则表、状态寄存器
 - 等价于有限状态机
- 移动、读取和写入
 - 依据：当前状态，输入符号，规则表
 - 移动：读写头位置
 - 读取：规则表 & 输入符号
 - 写入：符号，状态寄存器



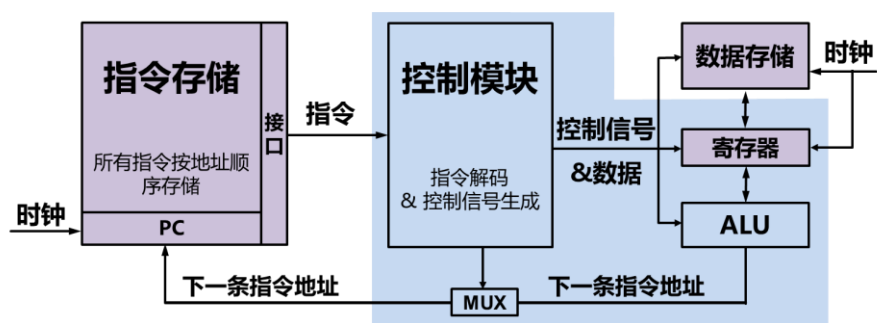
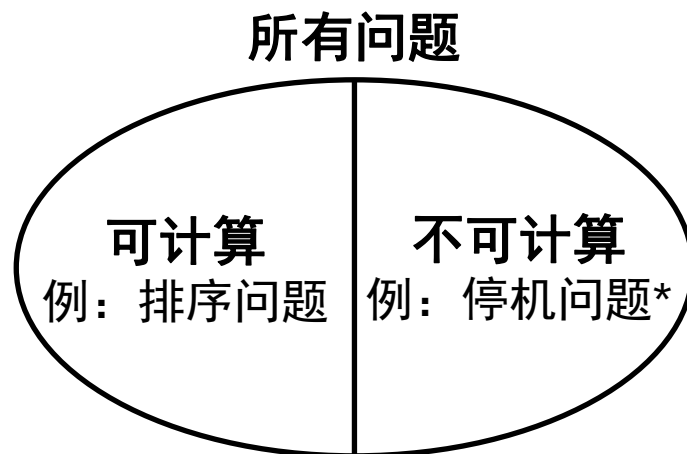
附录：图灵机

- 图灵机难以处理不同的任务
 - 不同任务：如何设计状态转移表？
- 等价图灵机模型
 - 普林斯顿架构
 - 哈佛架构

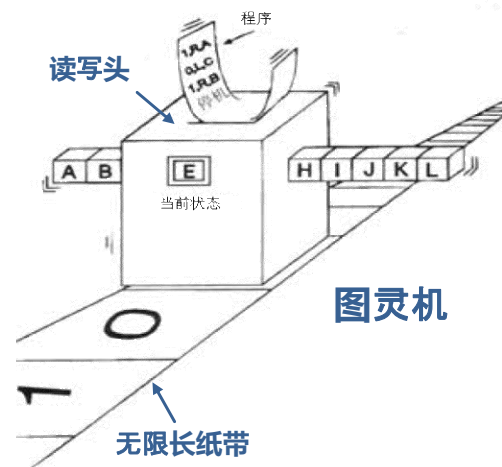


附录：可计算性与图灵完备

- 计算机不可能求解所有问题
 - 可计算性
 - 停机问题
- 如何判断电路能够支持通用算法
 - 图灵完备性



等价



图灵完备！

附录：编译 vs 解释

	COMPILER	INTERPRETER
Input	It takes an entire program at a time.	It takes a single line of code or instruction at a time.
Output	It generates intermediate object code.	It does not produce any intermediate object code.
Working mechanism	The compilation is done before execution.	Compilation and execution take place simultaneously.
Speed	Comparatively faster	Slower
Memory	Memory requirement is more due to the creation of object code.	It requires less memory as it does not create intermediate object code.
Errors	Display all errors after compilation, all at the same time.	Displays error of each line one by one.
Error detection	Difficult	Easier comparatively
Pertaining Programming languages	C, C++, C#, Scala, typescript uses compiler.	PHP, Perl, Python, Ruby uses an interpreter.